MEC-SETEC
INSTITUTO FEDERAL MINAS GERAIS - *Campus* Formiga
Curso de Ciência da Computação

PROTÓTIPO DE UM MODELADOR DE DIAGRAMAS ER COLABORATIVO

Wayne Nascimento Souza

Orientador: Roger Santos Ferreira

Formiga - MG

2026

WAYNE NASCIMENTO SOUZA

PROTÓTIPO DE UM MODELADOR DE DIAGRAMAS ER COLABORATIVO

Monografia do trabalho de conclusão de curso apresentado ao Instituto Federal Minas Gerais - Campus Formiga, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador: Roger Santos Ferreira

Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais

Campus Formiga

Ciência da Computação.

Formiga - MG

2026

Agradecimentos

Aos meus pais, que torceram por mim do início ao fim, e que, convenhamos, se preocuparam muito mais do que precisavam;

Aos meus colegas, companheiros de corredor, de café e de conversas que começavam sobre código e terminavam em qualquer outro assunto;

Aos meus professores, por entregarem muito mais do que conteúdo: uma base sólida que moldou o profissional que me tornei;

E a todos que, de alguma forma, cruzaram o meu caminho e deixaram um pouco de si nessa caminhada, mesmo que sem saber, fizeram parte desta jornada.

"Não se trata de provar que os outros estão errados, mas sim de provar que você está certo." (Sonya Curry)

Resumo

A modelagem de bancos de dados é uma etapa fundamental no desenvolvimento de sistemas de *software*, e os diagramas ER constituem uma ferramenta essencial para essa finalidade. No entanto, as soluções disponíveis no mercado dificilmente combinam colaboração em tempo real, modelagem interativa e acesso gratuito em uma única plataforma. Este trabalho apresenta o desenvolvimento de um protótipo de modelador de diagramas ER colaborativo, acessível pelo navegador, que permite a múltiplos usuários trabalharem simultaneamente no mesmo diagrama. O sistema foi construído sobre uma arquitetura cliente-servidor, com comunicação bidirecional em tempo real entre os colaboradores e mecanismos para o controle de acessos concorrentes. Adotou-se uma estratégia de persistência que separa o armazenamento de dados estruturados do armazenamento dos diagramas, adequando cada solução à natureza da informação que armazena. O protótipo oferece controle de acesso por perfis de usuário, autenticação por *token*, gerenciamento de projetos e equipes, e suporte à importação e exportação de definições de banco de dados. A comparação com ferramentas consolidadas no mercado evidenciou que, dentre as ferramentas analisadas, apenas o protótipo reúne, de forma gratuita, colaboração em tempo real e interoperabilidade com padrões amplamente utilizados. A validação combinou testes unitários, de integração e de sistema, teste de mutação e medição do tempo de propagação das alterações entre os colaboradores. Todos os casos de teste foram aprovados, inclusive os que reproduzem a edição simultânea do mesmo diagrama por usuários com perfis distintos. Conclui-se que é possível reunir, em uma única ferramenta *web* gratuita, a modelagem interativa de diagramas ER, a colaboração em tempo real e a importação e exportação de esquemas SQL.

Palavras-chave: diagrama entidade-relacionamento; modelagem de banco de dados; colaboração em tempo real; ferramenta *web*; desenvolvimento de *software*.

Abstract

Database modelling is a fundamental step in software system development, and ER diagrams constitute an essential tool for this purpose. However, available solutions rarely combine real-time collaboration, interactive modelling, and free access in a single platform. This work presents the development of a prototype collaborative ER diagram modeller, accessible through a web browser, that allows multiple users to work simultaneously on the same diagram. The system was built on a client-server architecture, with real-time bidirectional communication between collaborators and mechanisms for concurrent access control. A persistence strategy was adopted that separates the storage of structured data from diagram storage, matching each solution to the nature of the information it stores. The prototype offers role-based access control, token-based authentication, project and team management, and support for importing and exporting database definitions. A comparison with established market tools showed that, among the tools analysed, only the prototype combines, at no cost, real-time collaboration and interoperability with widely used standards. Validation combined unit, integration and system tests, mutation testing and measurement of the propagation time of changes between collaborators. All test cases passed, including those that reproduce simultaneous editing of the same diagram by users with different roles. It is concluded that interactive ER modelling, real-time collaboration and SQL schema import and export can be combined in a single free web tool.

Keywords: entity-relationship diagram; database modelling; real-time collaboration; web tool; software development.

Lista de ilustrações

Figura 1 – Diagrama de caso de uso do ator Viewer 35
Figura 2 – Diagrama de caso de uso do ator Editor destacando o fluxo principal . 36
Figura 3 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de tabelas 36
Figura 4 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de atributos 37
Figura 5 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de relacionamentos e a movimentação de elementos 37
Figura 6 – Diagrama de caso de uso do ator Editor destacando a importação SQL 38
Figura 7 – Diagrama de caso de uso do ator Owner destacando o *login* e a criação e listagem de projetos 38
Figura 8 – Diagrama de caso de uso do ator Owner destacando a gestão do projeto e do time 39
Figura 9 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de membros 39
Figura 10 – Diagrama de caso de uso do ator Owner destacando a visualização e a edição do diagrama 40
Figura 11 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de tabelas 40
Figura 12 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de atributos 41
Figura 13 – Diagrama de caso de uso do ator Owner destacando relacionamentos, movimentação de elementos e importação SQL 41
Figura 14 – Modelo conceitual do banco de dados 42
Figura 15 – Modelo lógico do banco de dados 43
Figura 16 – Esquema do banco de dados não-relacional 44
Figura 17 – Arquitetura do protótipo 48
Figura 18 – Arquitetura de autenticação e autorização do *back-end* 49
Figura 19 – Tela de *login* 55
Figura 20 – Tela principal com listagem de projetos 56
Figura 21 – *Modal* de criação de projeto 57
Figura 22 – *Modal* de edição de projeto e gerenciamento de time 58
Figura 23 – *Modal* de adição de membro ao projeto 59
Figura 24 – Editor de diagramas com entidades e relacionamentos 60
Figura 25 – *Modal* de edição de entidade 61
Figura 26 – Editor de diagramas com legenda de tipos de dados 62

Figura 27 – Editor de diagramas no perfil *Viewer* 63
Figura 28 – Entidade bloqueada por outro colaborador durante edição simultânea . 64
Figura 29 – Seleção do *script* SQL para importação 65
Figura 30 – Confirmação de importação bem-sucedida 66
Figura 31 – Diagrama reconstruído a partir do *script* SQL importado 67
Figura 32 – Salvamento do *script* SQL exportado 68

Lista de tabelas

Tabela 1 – Comparativo das Ferramentas 18
Tabela 2 – Níveis de teste do protótipo 29
Tabela 3 – Roteiro dos testes de sistema 31
Tabela 4 – Comparação entre as alternativas investigadas para a área de desenho . 47
Tabela 5 – Testes automatizados por nível 51
Tabela 6 – Relatório de cobertura de código dos testes do *back-end* 51
Tabela 7 – Relatório de cobertura de código dos testes do *front-end* 52
Tabela 8 – Resultado do teste de mutação 52
Tabela 9 – Resultado dos testes de sistema 53
Tabela 10 – Latência de propagação no nível do protocolo (ms) 54
Tabela 11 – Latência percebida pelo usuário (ms) 54
Tabela 12 – Comparação entre o protótipo e ferramentas de modelagem ER 70
Tabela 13 – Relação entre os objetivos específicos, os casos de uso e as evidências . 77

Lista de abreviaturas e siglas

API	*Application Programming Interface*, ou Interface de Programação de Aplicações
DDL	*Data Definition Language*, ou Linguagem de Definição de Dados
ER	*Entity-Relationship*, ou Entidade-Relacionamento
HTTP	*Hypertext Transfer Protocol*
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
MVC	*Model-View-Controller*, ou Modelo, Visão e Controle
REST	*Representational State Transfer*, ou Transferência de Estado Representacional
SGBD	Sistema de Gerenciamento de Banco de Dados
SPA	*Single Page Application*, ou Aplicação de Página Única
SQL	*Structured Query Language*, ou Linguagem de Consulta Estruturada
URL	*Uniform Resource Locator*
VCS	*Version Control System*, ou Sistema de Controle de Versão

Sumário

1 INTRODUÇÃO 13
1.1 Justificativa 14
1.2 Objetivos 15
1.2.1 Objetivo Geral 15
1.2.2 Objetivos Específicos 15

2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Ferramentas de Modelagem de Diagramas ER 17
2.1.1 brModeloWeb 17
2.1.2 SqlDBM 17
2.1.3 Vertabelo 17
2.2 Análise Comparativa 18

3 METODOLOGIA 19
3.1 Materiais 19
3.1.1 Equipamentos 19
3.1.2 Análise, Modelagem e Projeto 19
3.1.3 Persistência de dados 19
3.1.4 Implementação 20
3.1.5 Testes 21
3.2 Métodos 21
3.2.1 Abordagem Metodológica 21
3.2.1.1 Método Científico 22
3.2.1.2 Modelo em Espiral de Boehm 22
3.2.1.3 Metodologia Scrum 22
3.2.2 Análise e Projeto do Sistema 22
3.2.2.1 Arquitetura Cliente-Servidor 23
3.2.2.2 Protocolo de Comunicação em Tempo Real 23
3.2.3 Modelagem das Bases de Dados 23
3.2.3.1 Dados Relacionais 24
3.2.3.2 Dados Não Relacionais 24
3.2.4 Dialeto SQL 24
3.2.5 Padrão de Arquitetura de Software e de Código 24
3.2.5.1 Model-View-Controller 25
3.2.5.2 Clean Code 25
3.2.6 Estratégia de Controle de Versão 26

3.2.7 Autenticação e Autorização . . . 27
3.2.8 Estratégia de Concorrência . . . 28
3.2.9 Estratégia de Testes . . . 28
3.2.9.1 Testes Unitários . . . 29
3.2.9.2 Testes de Integração . . . 29
3.2.9.3 Testes de Sistema . . . 30
3.2.9.4 Teste de Mutação . . . 31
3.2.9.5 Medição de Desempenho . . . 32
3.2.10 Design da Interface do Usuário . . . 32
3.2.11 Idioma do Projeto . . . 33

4 RESULTADOS . . . 35
4.1 Diagrama de Caso de Uso . . . 35
4.2 Modelos de Dados . . . 41
4.3 Investigação da Biblioteca de Diagramação . . . 45
4.4 Arquitetura do Protótipo . . . 48
4.5 Funcionalidades Implementadas . . . 49
4.5.1 Autenticação e Autorização . . . 49
4.5.2 Gerenciamento de Projetos e Times . . . 49
4.5.3 Editor de Diagramas ER . . . 50
4.5.4 Colaboração em Tempo Real . . . 50
4.5.5 Importação e Exportação de Diagramas . . . 50
4.6 Testes e Validação do Protótipo . . . 50
4.6.1 Testes Automatizados e Cobertura de Código . . . 51
4.6.2 Teste de Mutação . . . 52
4.6.3 Testes de Sistema . . . 53
4.6.4 Desempenho da Colaboração em Tempo Real . . . 53
4.7 *Interface* do Usuário . . . 54
4.7.1 Autenticação . . . 54
4.7.2 Gerenciamento de Projetos e Times . . . 55
4.7.3 Editor de Diagramas . . . 59
4.7.4 Importação e Exportação SQL . . . 64
4.8 Disponibilidade do Código-Fonte e da Aplicação . . . 69

5 RESULTADOS E DISCUSSÕES . . . 70
5.1 Comparação com Ferramentas Existentes . . . 70
5.2 Avaliação dos Testes . . . 71
5.3 Avaliação do Desempenho . . . 72
5.4 Ameaças à Validade . . . 73
5.5 Dificuldades e Lições Aprendidas . . . 73

5.6 Limitações do Protótipo 74
5.7 Trabalhos Futuros 75
5.7.1 Camada de Cache Distribuído 75
5.7.2 Escalabilidade Horizontal do Canal WebSocket 75
5.7.3 Tratamento de Erros e Resiliência 76
5.7.4 Tempos de Resposta da Colaboração 76
5.7.5 Reforço dos Testes 76

6 CONSIDERAÇÕES FINAIS 77

REFERÊNCIAS 79

ANEXOS 84

ANEXO A – EXPANSÃO DO CASO DE USO 85

1 Introdução

O diagrama ER é uma representação gráfica usada para projetar e modelar um banco de dados. Ele auxilia na visualização das entidades (objetos, coisas ou conceitos) envolvidas em um sistema, bem como os relacionamentos entre essas entidades. Um diagrama ER é composto por entidades, atributos e relacionamentos. Segundo (ELMASRI; NAVATHE, 2017), as entidades são representadas por retângulos e correspondem a objetos do mundo real, como pessoa, lugar, objeto e assim por diante. Os atributos são características dessa entidade, como nome, idade, entre outros. Os relacionamentos são representados por linhas que mostram como a entidade está conectada com outras.

Existem diferentes tipos de relacionamentos em um diagrama ER, como relacionamento um-para-um, um-para-muitos e muitos-para-muitos. Esses relacionamentos ajudam a definir como as entidades estão associadas entre si. Além disso, é possível adicionar restrições e regras específicas ao diagrama, como chaves primária e estrangeira, que garantem a integridade dos registros no banco de dados.

Dada a rapidez com a qual os dados estão crescendo no cenário atual, a evolução da modelagem de um banco de dados tornou-se cada vez mais imperativa. Com o avanço da tecnologia e o aumento significativo na geração de dados, torna-se essencial que as ferramentas usadas para modelar esses bancos estejam alinhadas às demandas modernas.

No ambiente de desenvolvimento e projeto, a colaboração em tempo real tornou-se essencial, segundo (HSU et al., 2021). Isso permite uma maior interação e comunicação entre os membros da equipe, independentemente da distância física. Com as ferramentas adequadas, é possível compartilhar informações de maneira eficaz, colaborar na criação de conteúdo e acompanhar o progresso do projeto em tempo real.

Essas ferramentas colaborativas agilizam a troca de ideias, o trabalho em equipe e a resolução de problemas (INOUE et al., 2014). Permitem, por exemplo, a realização de reuniões virtuais, o debate de ideias, o compartilhamento de documentos e atualizações instantâneas. Com isso, membros da equipe permanecem alinhados e informados, evitando atrasos e retrabalhos.

Ademais, a colaboração em tempo real promove a transparência e a visibilidade do projeto, uma vez que todos os envolvidos têm acesso à informação e podem acompanhar o progresso das atividades. Esse dinamismo contribui para um ambiente de trabalho mais eficiente, minimizando custos e otimizando a produtividade, conforme (FUKS; RAPOSO; GEROSA, 2002).

Portanto, o desenvolvimento de um modelador de diagramas ER colaborativo não

é apenas oportuno, mas essencial para atender às demandas contemporâneas dos projetos de desenvolvimento de aplicações, garantindo que as soluções de banco de dados sejam robustas, eficientes e alinhadas com as necessidades de negócios em constante evolução.

1.1 Justificativa

Em um mundo digitalizado e interconectado, a excelência no projeto depende, em grande parte, da colaboração eficaz entre profissionais. A modelagem de um banco de dados, fundamental na arquitetura de um sistema, destaca-se nesse contexto. No entanto, o cenário atual revela uma carência de uma ferramenta acessível e colaborativa para a modelagem de diagramas ER. As alternativas disponíveis apresentam restrições, quer sejam de natureza funcional ou financeira. Por exemplo, algumas ferramentas, mesmo em seus pacotes mais elementares, possuem um custo elevado de aquisição e não disponibilizam todas as funcionalidades. Outras, em sua versão básica, não contemplam a opção para diagramas ER. Adicionalmente, a colaboração em tempo real, em certos casos, é limitada a um único usuário, relegando os demais a uma posição de mera visualização.

Diante da velocidade da inovação tecnológica e da complexidade crescente dos sistemas, uma ferramenta que permite a simultaneidade de contribuição de vários usuários é essencial. Ela não apenas potencializa o tempo de desenvolvimento, mas também favorece a identificação de inconsistências ou pontos de melhorias. Esta agilidade torna-se ainda mais crucial considerando a expansão da solução em nuvem e banco de dados distribuído, onde a colaboração remota é uma realidade inevitável.

O espectro dos beneficiários de tal ferramenta é vasto. Abrange desde um desenvolvedor ou arquiteto de sistema até o gestor de projeto, todos buscando um processo de modelagem mais ágil e coeso. Uma empresa com equipe dispersa geograficamente ou que opera sob metodologia ágil certamente verá um valor imensurável em uma plataforma que elimina barreira de comunicação e facilita a sinergia.

A longo prazo, a adoção de uma abordagem colaborativa na modelagem de diagramas ER tem o potencial de estabelecer um novo padrão na área. Com o reconhecimento de seu benefício, é provável que presenciemos uma transição para um método mais integrado, gerando transparência, eficácia e inovação. Ademais, à medida que uma tecnologia emergente, como a Inteligência Artificial, se integra ao processo, é plausível imaginar uma ferramenta que não apenas facilite a colaboração, mas também forneça sugestões e otimização automática, elevando a prática a um patamar inédito de sofisticação.

Considerando as demandas e desafios contemporâneos do mercado, o desenvolvimento de uma ferramenta colaborativa para modelagem de um diagrama ER não é apenas uma resposta adequada, mas uma iniciativa visionária, preparando o terreno para a necessidade futura da indústria.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver um protótipo de modelador de diagramas ER colaborativo para a modelagem de bancos de dados, permitindo contribuições simultâneas de diversos usuários.

1.2.2 Objetivos Específicos

1. Modelar os dados de tabelas, relacionamentos, atributos, tipos de dados, posicionamento dos elementos na tela e outros. Além disso, persistir os dados em uma base de dados após a conclusão de uma sequência de ações.

2. Investigar a biblioteca GoJS, compreendendo suas capacidades, limitações e possíveis aplicações no contexto do modelador de diagramas ER colaborativo proposto.

3. Desenvolver uma interface que permita a múltiplos usuários trabalharem simultaneamente no mesmo diagrama ER, visualizando atualizações em tempo real.

4. Desenvolver mecanismos de autenticação e autorização, garantindo que apenas usuários autorizados possam acessar e modificar os diagramas.

5. Desenvolver a importação e a exportação de diagramas ER por meio de *scripts* SQL, permitindo construir o diagrama a partir de um esquema de banco de dados existente e obter o esquema correspondente a um diagrama modelado.

2 Fundamentação Teórica

A modelagem de dados é a atividade que antecede a construção de um banco de dados e consiste em representar, de forma abstrata, os objetos de um domínio e as associações existentes entre eles. Conforme ELMASRI e NAVATHE (2017), essa atividade é organizada em três níveis: o conceitual, que descreve o domínio sem vinculação a uma tecnologia específica e tem no diagrama ER sua principal forma de representação; o lógico, que traduz esse modelo para as estruturas de um determinado tipo de banco de dados; e o físico, que define como os dados serão efetivamente armazenados. O diagrama ER ocupa, portanto, uma posição central no projeto de um sistema, pois é a partir dele que as demais decisões sobre o armazenamento dos dados são derivadas.

Além de orientar a construção do banco de dados, o diagrama ER cumpre um papel de comunicação: é sobre ele que analistas, desenvolvedores e demais envolvidos discutem o domínio e constroem um entendimento comum a respeito dele. Essa natureza compartilhada faz da modelagem uma atividade essencialmente coletiva e explica por que a possibilidade de várias pessoas trabalharem sobre um mesmo modelo é uma demanda recorrente nas ferramentas de apoio ao projeto de bancos de dados.

Os sistemas que dão suporte a esse tipo de trabalho coletivo são objeto de estudo da área de *groupware*. Segundo FUKS, RAPOSO e GEROSA (2002), tais sistemas se apoiam em três dimensões (comunicação, coordenação e cooperação), que precisam ser contempladas de maneira articulada: não basta permitir que os participantes atuem sobre um artefato comum, é necessário também coordenar essa atuação para que as contribuições não se anulem. No caso da edição simultânea, a coordenação se materializa em mecanismos de controle de concorrência, responsáveis por impedir que alterações realizadas ao mesmo tempo sobre um mesmo elemento conduzam o artefato a um estado inconsistente (FILIPPO, 2025).

Quando essas dimensões são atendidas, o ganho para a equipe é expressivo: a colaboração em tempo real encurta o ciclo de troca de ideias, oferece visibilidade imediata sobre a contribuição de cada participante e reduz o retrabalho decorrente de versões divergentes de um mesmo artefato (INOUE et al., 2014; HSU et al., 2021). São esses conceitos que orientam a análise das ferramentas de modelagem de diagramas ER apresentada nas seções seguintes.

2.1 Ferramentas de Modelagem de Diagramas ER

Esta seção analisa comparativamente três ferramentas de modelagem de diagramas ER disponíveis no mercado (brModeloWeb, SqlDBM e Vertabelo) com o objetivo de identificar seus pontos fortes e fracos em relação à modelagem colaborativa. As ferramentas foram selecionadas a partir de dois critérios: serem aplicações *web*, executadas diretamente no navegador, e oferecerem suporte à modelagem de diagramas ER.

2.1.1 brModeloWeb

O brModeloWeb é uma ferramenta de modelagem de dados *online*, gratuita e de código aberto, mantida pelo Grupo de Banco de Dados da Universidade Federal de Santa Catarina e amplamente utilizada em ambientes acadêmicos e de ensino (brModeloWeb, 2026; NETO; MELLO, 2026). A capacidade de gerar *scripts* SQL a partir do modelo lógico a torna uma opção atraente para iniciantes e para projetos de menor escala. No entanto, em termos de funcionalidades colaborativas, o brModeloWeb apresenta limitações. Embora a sua versão hospedada permita o compartilhamento de diagramas, a ferramenta não oferece suporte à edição simultânea por múltiplos usuários nem controle de versão integrado. Sendo assim, a colaboração se resume ao compartilhamento dos modelos, o que não permite a rastreabilidade das alterações nem o trabalho conjunto em tempo real.

2.1.2 SqlDBM

O SqlDBM é uma solução comercial de modelagem de banco de dados *online*, com foco na colaboração e na integração com diferentes SGBDs (SqlDBM, 2026). A ferramenta permite que múltiplos usuários trabalhem simultaneamente sobre o mesmo modelo, além de oferecer comentários entre os membros da equipe e fluxos de aprovação das alterações. Adicionalmente, o SqlDBM oferece controle de versão integrado, permitindo comparar versões do modelo e reverter alterações, e a integração com repositórios *git*, como GitHub e Azure DevOps, permite que as equipes gerenciem seus modelos de dados da mesma forma que gerenciam o código-fonte da aplicação.

2.1.3 Vertabelo

O Vertabelo é uma ferramenta de modelagem de dados *online* que engloba desde a modelagem conceitual (diagramas ER) até a modelagem lógica e física de bancos de dados. Em 2025, a ferramenta foi adquirida pela Redgate e passou a ser comercializada sob o nome Redgate Data Modeler (Redgate Software, 2026). Similarmente ao SqlDBM, oferece um ambiente compartilhado no navegador com colaboração em tempo real, permitindo que múltiplos usuários criem e modifiquem os modelos de dados simultaneamente. A ferramenta conta ainda com versionamento e acompanhamento das alterações, validação dos modelos

durante a edição e geração de *scripts* SQL a partir do modelo. A sua comercialização ocorre por assinatura anual por usuário, sem plano gratuito, havendo apenas um período de avaliação de 14 dias.

2.2 Análise Comparativa

A análise das ferramentas brModeloWeb, SqlDBM e Vertabelo revela diferentes abordagens para a modelagem ER colaborativa. Enquanto o brModeloWeb oferece uma solução mais simples e focada em modelagem básica, o SqlDBM e o Vertabelo apresentam funcionalidades mais avançadas de colaboração, como edição em tempo real e controle de versão.

Funcionalidade	**brModeloWeb**	**SqlDBM**	**Vertabelo**
Colaboração em Tempo Real	Não	Sim	Sim
Controle de Versão	Não	Sim	Sim
Geração de SQL	Sim	Sim	Sim
Plano gratuito	Sim	Não	Não

Tabela 1 – Comparativo das Ferramentas

3 Metodologia

Este capítulo descreve como o protótipo de modelador de diagramas ER colaborativo foi concebido, desenvolvido e validado. Ele está organizado em duas partes. A primeira, de Materiais, reúne os equipamentos, as ferramentas, as linguagens e as bibliotecas empregados ao longo do trabalho. A segunda, de Métodos, apresenta a abordagem metodológica adotada na condução do projeto e as decisões de análise e projeto que orientaram a implementação, como a arquitetura do sistema, o protocolo de comunicação em tempo real, a modelagem das bases de dados, a estratégia de controle de versão, os mecanismos de autenticação e autorização e a estratégia de concorrência.

3.1 Materiais

Nesta seção, são listados os equipamentos de *hardware* e as ferramentas, linguagens, bibliotecas e plataformas de *software* empregados na concepção, no desenvolvimento e na validação da aplicação, de modo a permitir a reprodução do ambiente de trabalho. As decisões de arquitetura que motivaram a escolha dos itens mais relevantes são discutidas e justificadas na seção de Métodos, a seguir.

3.1.1 Equipamentos

- Notebook: processador Intel Core i5, 1.6 GHz, 8 GB de memória RAM e SSD de 240 GB, com *dual boot* para os sistemas operacionais Windows 10 e Linux Mint.
- PC: processador AMD Ryzen 5 5600G, 3.9 GHz, 32 GB de memória RAM e SSD de 477 GB, com sistema operacional Windows 11.

3.1.2 Análise, Modelagem e Projeto

- Unified Modeling Language (UML) (VIEIRA, 2015): elaboração dos diagramas de caso de uso do sistema.
- Excalidraw, com a biblioteca *Shapes for UML and ER Diagrams*: criação dos diagramas de arquitetura e dos modelos de dados.

3.1.3 Persistência de dados

- PostgreSQL (PostgreSQL Global Development Group, 2024): persistência dos dados relacionais (usuários, projetos e permissões de acesso).

- MongoDB (IBM, 2021): persistência da estrutura dos diagramas ER.
- DBeaver: administração, consulta e manipulação dos dados no PostgreSQL e no MongoDB.

3.1.4 Implementação

- Git (Git, 2025): controle de versão do código-fonte.
- GitHub (GitHub, 2024a): hospedagem e repositório central do código-fonte.
- GitHub Copilot (GitHub Copilot, 2026): apoio à revisão de código em parte dos *pull requests*.
- Docker (Docker, 2024): containerização do ambiente de desenvolvimento e dos bancos de dados.
- IntelliJ IDEA (JetBrains, 2024a): IDE para o desenvolvimento do *back-end* em Java e Spring.
- WebStorm (JetBrains, 2024b): IDE para o desenvolvimento do *front-end* em Angular.
- Java (Oracle, 2024a): linguagem de programação do *back-end*.
- Spring Boot (VMware, 2024a): *framework* de estruturação do *back-end* e das APIs REST.
- Spring WebSocket (VMware, 2024e): comunicação em tempo real entre *back-end* e clientes.
- Spring Security (VMware, 2024d): autenticação e autorização da aplicação, em conjunto com JWT.
- Spring Data JPA (VMware, 2024b): camada de acesso a dados do PostgreSQL.
- Spring Data MongoDB (VMware, 2024c): camada de acesso a dados do MongoDB.
- Angular (Google, 2022): construção da interface do usuário como *Single-Page Application*.
- GoJS (Northwoods Software Corporation, 2024): renderização e manipulação interativa dos diagramas ER.
- stompjs e SockJS (StompJS, 2023): comunicação via *WebSocket* no lado do cliente.
- Node Package Manager (npm) (GitHub, 2024b): gestão das dependências do *front-end*.

3.1.5 Testes

- JUnit (JUnit, 2023) e Mockito (Mockito, 2025): testes unitários da lógica de negócio no *back-end*.
- Spring Boot (VMware, 2024a) e H2 (H2 Database Engine, 2024): testes de integração do *back-end*, que executam a aplicação completa sobre um banco de dados relacional mantido em memória.
- Jasmine (Jasmine, 2024) e Karma (Karma, 2024): testes unitários dos componentes e serviços do *front-end*.
- Playwright (Microsoft, 2024): testes de sistema, executados no navegador Google Chrome com sessões simultâneas de usuários distintos.
- JaCoCo (JaCoCo, 2024): medição da cobertura de código dos testes unitários do *back-end*.
- PIT (PIT, 2024) e Stryker (Stryker Mutator, 2024): teste de mutação do *back-end* e do *front-end*, respectivamente.
- SonarQube (SonarSource, 2024): análise estática de qualidade e cobertura de código do *back-end*.
- Postman (Postman, 2024): testes manuais das APIs REST do *back-end*.
- Google Chrome (Google, 2025) e Mozilla Firefox (Mozilla, 2026): testes manuais de compatibilidade da interface do usuário.

3.2 Métodos

Nesta seção, são apresentados os métodos e as abordagens que nortearam o planejamento, a análise, o projeto e a implementação da aplicação. A condução do trabalho foi baseada em uma combinação de práticas conhecidas da Engenharia de *Software* e da pesquisa científica, adaptadas para atender suas especificidades.

3.2.1 Abordagem Metodológica

Para a condução deste projeto, adotou-se uma abordagem metodológica híbrida, que integra conceitos do método científico, elementos do modelo em espiral de Boehm e práticas da metodologia Scrum.

3.2.1.1 Método Científico

O método científico serviu como alicerce estrutural do trabalho, definindo as etapas macro de observação de um problema (a necessidade de uma ferramenta de modelagem ER colaborativa em tempo real), formulação de uma hipótese (a de que é possível reunir, em uma única ferramenta gratuita, a modelagem interativa de diagramas ER, a edição simultânea do mesmo diagrama por múltiplos usuários com papéis de acesso distintos e a importação e exportação de esquemas SQL, superando as limitações de custo, de suporte a diagramas ER e de restrição da edição a um único usuário identificadas nas ferramentas semelhantes), experimentação (o desenvolvimento da solução) e análise dos resultados.

3.2.1.2 Modelo em Espiral de Boehm

Proposto por BOEHM (1988), o modelo em espiral organiza o desenvolvimento em ciclos sucessivos, nos quais cada volta da espiral é orientada pela identificação e mitigação dos riscos do projeto (PRESSMAN; MAXIM, 2021). O modelo foi empregado conceitualmente para guiar o ciclo de vida do projeto de forma iterativa e incremental. Cada ciclo envolveu planejamento, análise de riscos (como a complexidade da sincronização em tempo real), engenharia (codificação e testes) e avaliação pelo desenvolvedor, permitindo o refinamento contínuo da aplicação a cada iteração.

3.2.1.3 Metodologia Scrum

O Scrum é um *framework* ágil, definido por SCHWABER e SUTHERLAND (2020), que organiza o trabalho em ciclos curtos e de duração fixa, denominados *sprints*, a partir de uma lista priorizada de itens a serem desenvolvidos, o *Product Backlog*. A metodologia foi utilizada de forma adaptada para o gerenciamento das atividades de desenvolvimento. O trabalho foi decomposto em épicos, representando as grandes funcionalidades, que por sua vez foram detalhados em tarefas menores e mais específicas, facilitando o acompanhamento do progresso e a organização do trabalho em ciclos curtos de desenvolvimento.

3.2.2 Análise e Projeto do Sistema

A fase de análise e projeto teve como objetivo delimitar o escopo funcional do protótipo e orientar as decisões técnicas de sua construção. Esse escopo foi definido a partir de duas fontes: os objetivos específicos estabelecidos na introdução e a análise comparativa apresentada na fundamentação teórica. Da análise comparativa, extraíram-se tanto as funcionalidades comuns às ferramentas analisadas, como a modelagem de diagramas ER e a exportação de *scripts* SQL, quanto as limitações que o protótipo buscou superar, como a ausência de edição simultânea e de importação SQL no brModeloWeb e a exigência de assinatura paga no SqlDBM e no Vertabelo. Com base nessas fontes, foram elaborados diagramas de Caso de Uso. Essa técnica da *Unified Modeling Language* (UML) permitiu

identificar os atores e suas interações com o sistema, delimitando as funcionalidades a serem implementadas, como criar um projeto, convidar colaboradores, adicionar uma entidade ao diagrama, entre outras. A partir dessa análise, foram tomadas decisões arquiteturais cruciais.

3.2.2.1 Arquitetura Cliente-Servidor

Optou-se pela arquitetura macro do tipo cliente-servidor, padrão para aplicações *web* modernas. Nessa arquitetura, o *front-end* (cliente), executado no navegador do usuário, é responsável pela interface e interação, enquanto o *back-end* (servidor) centraliza a lógica de negócio, o controle de acesso e a persistência dos dados.

3.2.2.2 Protocolo de Comunicação em Tempo Real

Para atender à necessidade central de colaboração em tempo real, o protocolo WebSocket foi escolhido para a comunicação contínua entre cliente e servidor. A comunicação síncrona do protocolo HTTP, baseada em um ciclo de requisição e resposta estritamente iniciado pelo cliente, mostra-se ineficiente para este cenário. Soluções paliativas que utilizam HTTP, como o *polling* (onde o cliente consulta o servidor em intervalos regulares), geram uma alta latência e um tráfego de rede excessivo, pois cada consulta reinicia uma conexão e carrega metadados que são, na maioria das vezes, desnecessários.

O WebSocket, em contrapartida, estabelece uma conexão única, persistente e bidirecional após um *handshake* inicial. Essa arquitetura permite que o servidor envie atualizações aos clientes de forma proativa e instantânea (*server push*), com baixa sobrecarga de dados. Essa característica é fundamental para a viabilidade de sistemas colaborativos, onde a propagação de alterações deve ter a menor latência possível para garantir uma experiência de uso fluida e consistente entre os múltiplos usuários. Conforme SOUZA (2023), em aplicações que demandam alta interatividade e troca constante de pequenos pacotes de dados, o WebSocket apresenta um desempenho superior e um custo computacional significativamente menor quando comparado às abordagens baseadas em HTTP.

3.2.3 Modelagem das Bases de Dados

A estratégia de persistência de dados adotada neste projeto foi a abordagem híbrida, também conhecida como persistência poliglota (*polyglot persistence*). Essa decisão consistiu em combinar um sistema de gerenciamento de banco de dados relacional e um não relacional, a fim de utilizar a tecnologia mais adequada para cada tipo de dado manipulado pela aplicação, melhorando tanto o desempenho quanto a integridade da informação.

3.2.3.1 Dados Relacionais

Para o armazenamento de dados com estrutura bem definida e que exigem integridade referencial, como as informações de usuários, projetos e credenciais de acesso, optou-se pelo SGBD PostgreSQL. Sua conformidade com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) e seu sistema de controle de transações foram fatores determinantes para garantir a consistência e a segurança dos dados do sistema. A modelagem dessas informações foi formalizada por meio de um diagrama Entidade-Relacionamento (ER).

3.2.3.2 Dados Não Relacionais

Em contrapartida, para a persistência dos dados que compõem os diagramas ER do protótipo (como entidades, atributos, relacionamentos e suas respectivas posições na área de desenho) a escolha recaiu sobre o MongoDB. A natureza colaborativa e em tempo real da aplicação impõe uma necessidade de destaque para operações de leitura e escrita concorrentes, onde múltiplos usuários podem consultar e alterar o mesmo diagrama. Esta decisão é respaldada por estudos que demonstram a superioridade do MongoDB em relação ao PostgreSQL (POLITOWSKI; MARAN, 2014). O modelo de documentos permite que a estrutura de um diagrama seja armazenada de forma coesa e hierárquica, otimizando as consultas e simplificando a lógica de sincronização necessária para a colaboração em tempo real.

3.2.4 Dialeto SQL

Uma das funcionalidades centrais do protótipo é a capacidade de exportar e importar o diagrama ER como um *script* SQL. Para viabilizar essas operações, foi necessário definir um dialeto SQL de referência, já que diferentes SGBDs possuem particularidades sintáticas próprias em suas instruções DDL.

O dialeto adotado foi o MySQL. O MySQL é um sistema de gerenciamento de banco de dados relacional de código aberto, mantido pela Oracle Corporation, e é amplamente reconhecido como um dos SGBDs mais utilizados no mundo para aplicações *web* (Oracle, 2024b). Sua sintaxe DDL é amplamente difundida no mercado de desenvolvimento *web*, o que torna o dialeto MySQL uma escolha adequada para maximizar a interoperabilidade dos *scripts* gerados e importados pelo protótipo com outros ambientes e ferramentas do ecossistema de desenvolvimento de *software*.

3.2.5 Padrão de Arquitetura de Software e de Código

Com a arquitetura macro definida, foram adotados padrões específicos para a organização interna do código, tanto no *back-end* quanto no *front-end*.

3.2.5.1 Model-View-Controller

A escolha do padrão de arquitetura *Model-View-Controller* (MVC) foi uma decisão estratégica para a organização do código-fonte, sendo aplicada tanto no *back-end* quanto no *front-end* da aplicação. Este padrão promove a separação de responsabilidades, dividindo a aplicação em três componentes interconectados, o que foi útil para gerenciar a complexidade do sistema.

- *Model*: Camada de manipulação dos dados.
- *View*: Camada de interação do usuário.
- *Controller*: Camada de controle, responsável por ligar o *model* e a *view*, fazendo com que os *models* possam ser repassados para as *views* e vice-versa.

A adoção do padrão MVC em ambas as camadas do sistema não foi uma mera formalidade, mas uma escolha deliberada para obter vantagens concretas no processo de desenvolvimento. Essa abordagem está alinhada com as melhores práticas da Engenharia de *Software*, que destacam o MVC como um padrão que promove a coesão e o baixo acoplamento entre as partes de um sistema. Conforme aponta a literatura, a separação de interesses facilita o desenvolvimento paralelo, a reutilização de componentes e, crucialmente, a testabilidade e a manutenção do *software* (PRESSMAN; MAXIM, 2021).

- *Back-end*: O *framework* Spring implementa nativamente uma variação do MVC. As classes que usam uma determinada anotação atuam como Controladores, recebendo requisições HTTP. A lógica de negócio e os objetos de domínio representam o Modelo. A Visão é materializada pela resposta serializada, tipicamente em formato JSON, enviada ao cliente.
- *Front-end*: O *framework* Angular organiza a aplicação de forma análoga. Os componentes são formados por um *template* HTML, uma classe que contém a lógica e o estado, e os serviços que gerenciam os dados da aplicação.

3.2.5.2 Clean Code

A qualidade interna do código-fonte foi uma adotada durante todo o ciclo de desenvolvimento. Para tanto, foram escolhidos de forma sistemática os princípios de Código Limpo, conforme popularizados por MARTIN (2009), com o objetivo de produzir uma base de código legível e manutenível. A aplicação desses princípios se manifestou principalmente nas seguintes práticas:

- Nomes de variáveis e métodos significativos

- Métodos pequenos e com responsabilidade única
- Código autoexplicativo

O primeiro princípio foi aplicado promovendo atenção especial à nomenclatura para que o propósito de cada elemento do código fosse explícito. Em vez de nomes genéricos, utilizaram-se identificadores descritivos. Da mesma forma, os métodos foram nomeados para expressar claramente sua ação e seu resultado. Essa prática elimina a ambiguidade e torna o código mais intuitivo.

O segundo princípio foi aplicado na decomposição da lógica de negócio. Métodos e funções foram projetados para executar uma única tarefa e fazê-la bem. Por exemplo, um controlador de API REST que recebe uma requisição para adicionar uma nova entidade ao diagrama não executa a validação, a persistência no banco e a notificação via WebSocket dentro de um único método monolítico. Em vez disso, ele delega cada uma dessas responsabilidades a camadas de serviço distintas: um método para validar os dados, um para orquestrar a lógica de negócio e a persistência, e outro para notificar os clientes conectados. Essa separação torna o sistema mais modular, testável e fácil de depurar.

O terceiro princípio foi majoritariamente empregado na escrita de um código que, por sua própria estrutura e clareza, documenta seu funcionamento, minimizando a necessidade de comentários explicativos. A combinação de nomes significativos e funções com responsabilidade única resultou em um código que se assemelha a uma narrativa legível. Comentários foram utilizados de forma criteriosa, reservados para justificar decisões de projeto não triviais ou para explicar algoritmos complexos, em vez de parafrasear o que uma linha de código já expressa claramente.

3.2.6 Estratégia de Controle de Versão

A gestão do código-fonte é um pilar para a qualidade e a rastreabilidade de qualquer projeto de software. Para este trabalho, foi empregado o Sistema de Controle de Versão (VCS) *git*, com o código hospedado na plataforma GitHub (AQUILES; FERREIRA, 2014). A estratégia adotada foi um modelo híbrido, que combina a agilidade do *Trunk-Based Development* como fluxo principal, complementado por uma adaptação simplificada do *GitFlow* para o desenvolvimento de funcionalidades específicas.

A abordagem primária do *Trunk-Based Development*, na qual as alterações são integradas diretamente na ramificação principal (master), foi uma escolha deliberada para promover um ritmo de desenvolvimento mais acelerado e simplificar o gerenciamento de versões. A viabilidade dessa abordagem foi sustentada pelo contexto do projeto, que conta com um único desenvolvedor. Essa decisão encontra respaldo no estudo de LOPES et al. (2025), que aponta que a abordagem *Trunk-Based* é particularmente eficaz em projetos

de ritmo acelerado, com equipes menores e mais experientes, onde a complexidade de gerenciamento de múltiplas ramificações se torna um *overhead* desnecessário.

Para o desenvolvimento de funcionalidades mais complexas ou para a correção de *bugs* que exigissem um trabalho mais extenso, foi adotada uma adaptação do modelo *GitFlow*. Neste fluxo simplificado, uma ramificação de funcionalidade (*feature branch*) era criada a partir da master. Esta prática permitiu implementar e testar novas capacidades de forma isolada, sem comprometer a estabilidade da ramificação principal. Após a conclusão e validação, a *feature branch* era integrada de volta à master. Esse isolamento facilitou a organização das tarefas e garantiu uma camada adicional de validação, além de aprimorar a rastreabilidade do histórico de desenvolvimento, uma vez que cada conjunto de alterações significativas fica contido em sua própria ramificação (RIVERO et al., 2021).

A combinação dessas duas estratégias proporcionou o melhor de ambos os mundos para o escopo deste trabalho: a velocidade e a simplicidade do *Trunk-Based Development* para o progresso diário e a segurança e organização das ramificações de funcionalidade para as implementações mais críticas.

3.2.7 Autenticação e Autorização

A segurança da aplicação foi implementada utilizando uma combinação de tecnologias e estratégias, visando proteger os dados e garantir o acesso adequado aos recursos. O Spring Security, um *framework* utilizado no ecossistema Java, foi o pilar central da autenticação e autorização (VMware, 2024d).

Para a autenticação, foi adotado o padrão JWT. Após a autenticação bem-sucedida do usuário (verificação de credenciais), o servidor gera um *token* JWT, que é então enviado ao cliente. Este *token* contém informações sobre o usuário autenticado e suas permissões.

Os *tokens* JWT foram armazenados em *cookies* HTTP no cliente. Essa abordagem permite que o navegador gerencie automaticamente o envio do *token* em cada requisição subsequente ao servidor, simplificando a implementação e melhorando a segurança em relação ao armazenamento em *localStorage*, que é mais vulnerável a ataques XSS (*Cross-Site Scripting*) (BULGAKOVA; ZOSIMOV; POPRAVKIN, 2020).

No *back-end*, o Spring Security intercepta todas as requisições, validando a autenticidade do *token* JWT presente no *cookie*. Caso o *token* seja válido, o Spring Security extrai as informações do usuário e suas permissões, permitindo o acesso aos recursos autorizados.

No *front-end*, Angular Guards atuam para proteger rotas e componentes, impedindo o acesso não autorizado a determinadas áreas da aplicação, de acordo com OLIVEIRA, SANTANA e REIS (2024). Por exemplo, apenas usuários com ua determinada permissão podem acessar a página de gerenciamento de usuários.

Essa abordagem em camadas, combinando Spring Security, JWT, *Cookies* e Angular Guards, oferece uma solução de segurança, protegendo a aplicação contra acesso não autorizado e garantindo a integridade dos dados.

3.2.8 Estratégia de Concorrência

A colaboração em tempo real exige uma estratégia para as alterações simultâneas: sem ela, duas alterações feitas ao mesmo tempo podem entrar em conflito, e uma delas pode se perder. O protótipo combina dois mecanismos.

O primeiro é um mecanismo de *locks* (travas) por tabela, mantido em memória no servidor (HIPÓLITO, 2024). Ao abrir uma tabela para edição, o usuário adquire a trava daquela tabela, e os demais colaboradores ficam impedidos de editá-la até que ela seja liberada. A verificação de que a tabela está livre e a aquisição da trava são feitas em uma única operação atômica, de modo que dois pedidos simultâneos pela mesma tabela não podem ser atendidos ao mesmo tempo. A trava é liberada quando o usuário encerra a edição ou quando a sua conexão com o servidor termina e, nos dois casos, os demais colaboradores são notificados. Para que uma trava esquecida não bloqueie os demais indefinidamente, o servidor verifica as travas existentes a cada 2 minutos e libera as que foram adquiridas há mais de 5 minutos (FILIPPO, 2025).

O segundo mecanismo trata da gravação das alterações. Em vez de enviar o diagrama inteiro, cada cliente envia apenas os elementos que alterou desde o último estado recebido do servidor: as tabelas e os relacionamentos incluídos ou modificados e os identificadores dos removidos. O servidor aplica essas alterações elemento a elemento sobre o diagrama armazenado, uma de cada vez em cada projeto, e envia o resultado a todos os colaboradores. Ao receber um novo estado, cada cliente reaplica sobre ele as próprias alterações que ainda não foram gravadas. Dessa forma, alterações simultâneas em elementos diferentes são todas preservadas. Alterações simultâneas em um mesmo elemento seguem a regra de que a última gravação prevalece; na edição de tabelas, a trava descrita acima impede que elas ocorram.

As alterações de cada cliente são enviadas automaticamente depois de 1,5 segundo sem novas alterações. Esse intervalo agrupa em um único envio as alterações feitas em sequência, como as posições intermediárias de um elemento que está sendo arrastado, ao custo de atrasar na mesma medida a chegada da alteração aos demais colaboradores.

3.2.9 Estratégia de Testes

A validação do protótipo foi organizada em três níveis de teste, que verificam o sistema em escalas crescentes (PRESSMAN; MAXIM, 2021), complementados por uma avaliação da eficácia dos testes unitários e por uma medição de desempenho da colaboração

em tempo real. A Tabela 2 resume o que cada nível verifica, como os seus casos foram definidos e o que é exigido para considerá-lo aprovado.

Tabela 2 – Níveis de teste do protótipo

Nível	**O que verifica**	**Definição dos casos**	**Critério de aprovação**
Unitário	Cada classe do *back-end* e cada componente ou serviço do *front-end*, isoladamente	Escritos unidade a unidade, com a cobertura de código como critério de completude	Todos os testes aprovados e cobertura de 100%
Integração	O canal de comunicação em tempo real, com a aplicação completa em execução	Um caso para cada regra de acesso e de propagação do canal	Todos os casos aprovados
Sistema	O comportamento percebido pelos usuários, em navegadores reais	Roteiro derivado dos casos de uso do Anexo A	Resultado observado igual ao esperado em todos os casos, em três execuções

Fonte: Elaborado pelo autor

3.2.9.1 Testes Unitários

Os testes unitários verificam cada unidade de código de forma isolada: as dependências externas a ela, como os bancos de dados e a comunicação em tempo real, são substituídas por versões simuladas. Os casos desse nível não foram derivados dos casos de uso. Eles foram escritos unidade a unidade, com o objetivo de exercitar todos os caminhos de execução de cada uma, e por isso a cobertura de código foi adotada como critério de completude: o repositório rejeita qualquer atualização que reduza a cobertura abaixo de 100%. No *front-end*, o serviço de renderização dos diagramas e o componente que abre a conexão com o servidor, que dependem diretamente do GoJS e do SockJS, não fazem parte da medição. Como a cobertura indica apenas quais trechos foram executados, e não se o comportamento deles foi verificado (INOZEMTSEVA; HOLMES, 2014), a eficácia desses testes foi avaliada à parte, pelo teste de mutação.

3.2.9.2 Testes de Integração

Por substituírem as dependências de cada unidade, os testes unitários não verificam se as partes do sistema funcionam em conjunto. Para o canal de comunicação em tempo real, núcleo da colaboração, foram escritos testes de integração que executam a aplicação completa: um cliente real se conecta ao servidor pelo protocolo WebSocket, apresentando

o *token* JWT da mesma forma que o navegador, e as permissões de cada usuário são lidas de um banco de dados relacional mantido em memória. Somente o banco de dados dos diagramas é substituído por uma versão simulada. Foram definidos cinco casos, que verificam as seguintes regras:

- Uma alteração feita por um *Editor* chega aos demais membros do projeto.
- Uma alteração enviada por um *Viewer* não é gravada nem propagada.
- Uma conexão sem *token* válido é recusada.
- Um usuário de fora do projeto não recebe as alterações feitas nele.
- Alterações simultâneas de dois usuários em tabelas diferentes são ambas preservadas.

3.2.9.3 Testes de Sistema

Os testes de sistema verificam o protótipo da forma como os usuários o utilizam. Cada caso foi derivado de um caso de uso do Anexo A, com prioridade para os que envolvem mais de um usuário, já que a colaboração é o objetivo central do trabalho: a visualização e a edição do diagrama, em especial a regra de que as alterações são propagadas aos demais usuários conectados ao projeto, e a inclusão e a edição de tabelas. Em cada caso, o Google Chrome é aberto em sessões independentes, uma para cada membro do projeto, cada uma autenticada pela tela de *login*; as ações são executadas em uma sessão, e o resultado é verificado nas demais. Cada caso usa um projeto próprio, e as tabelas de que ele precisa como pré-condição são criadas antes da abertura dos navegadores, pela importação de um *script* SQL. Assim, cada caso depende apenas do comportamento que verifica. O roteiro, apresentado na Tabela 3, é executado automaticamente pelo Playwright.

Tabela 3 – Roteiro dos testes de sistema

ID	**Caso de uso**	**Pré-condição**	**Ações**	**Resultado esperado**
CT-01	Editar Diagrama Adicionar Tabela	Projeto vazio, aberto pelo *Owner*, pelo *Editor* e pelo *Viewer*	O *Owner* adiciona uma tabela	A tabela aparece para o *Editor* e para o *Viewer*
CT-02	Editar Diagrama Editar Tabela	Projeto com uma tabela, aberto pelo *Owner* e pelo *Editor*	O *Editor* abre a tabela e altera o seu nome	O novo nome aparece para o *Owner*
CT-03	Editar Tabela	Projeto com uma tabela, aberto pelo *Owner* e pelo *Editor*	O *Editor* abre a tabela para edição; o *Owner* tenta abri-la; o *Editor* encerra a edição; o *Owner* tenta de novo	A primeira tentativa é recusada, com a indicação de quem está editando; a segunda é permitida
CT-04	Visualizar Diagrama	Projeto com uma tabela, aberto pelo *Owner* e pelo *Viewer*	O *Viewer* procura a opção de adicionar tabela e tenta abrir a tabela para edição	A opção não é exibida, a edição é recusada e o diagrama do *Owner* não se altera
CT-05	Editar Diagrama Editar Tabela	Projeto com duas tabelas, aberto pelo *Owner* e pelo *Editor*	Cada um renomeia uma tabela diferente, e os dois salvam dentro do mesmo intervalo de 1,5 segundo	Ao reabrir o projeto, os dois usuários veem os dois novos nomes
CT-06	Visualizar Diagrama Adicionar Tabela	Projeto vazio, aberto pelo *Owner*	O *Owner* adiciona duas tabelas e recarrega a página	As duas tabelas continuam no diagrama
CT-07	Editar Tabela	Projeto com uma tabela, aberto pelo *Owner* e pelo *Editor*	O *Editor* abre a tabela para edição e fecha o navegador sem encerrar a edição	O *Owner* consegue abrir a tabela para edição

Fonte: Elaborado pelo autor

O roteiro foi executado três vezes consecutivas em cada um dos ambientes descritos na Subseção 3.2.9.5, o local e o publicado, para descartar resultados que dependessem do tempo de execução. No caso CT-07, como o servidor só libera a tabela quando percebe que a conexão do *Editor* terminou, o *Owner* repete a tentativa de abri-la a cada segundo, por até 60 segundos, e o tempo até a liberação é registrado.

3.2.9.4 Teste de Mutação

O teste de mutação avalia a capacidade dos testes de detectar defeitos (DEMILLO; LIPTON; SAYWARD, 1978). A ferramenta gera cópias do programa, cada uma com uma única alteração pequena no código, chamada de mutante, como a inversão de uma condição, a remoção de uma chamada ou a troca de um valor de retorno, e executa os testes sobre cada mutante. Se algum teste falha, o mutante é considerado morto, ou seja, a alteração foi detectada. Se todos passam, o mutante sobreviveu, o que indica um comportamento que os testes executam, mas não verificam. O escore de mutação é a proporção de mutantes mortos em relação ao total gerado. Foram usados o PIT no *back-end* e o Stryker no

front-end, com os operadores de mutação padrão de cada ferramenta, sobre os mesmos arquivos considerados na medição de cobertura. No *back-end*, os testes de integração não participaram da análise: como abrem conexões reais e aguardam intervalos de tempo, tornariam a execução de cada mutante muito mais longa.

3.2.9.5 Medição de Desempenho

O desempenho da colaboração em tempo real foi medido pelo tempo que uma alteração leva para chegar aos demais colaboradores, sob duas perspectivas.

A primeira é a latência de propagação, medida no nível do protocolo. Um *script* de medição, escrito em JavaScript com a mesma biblioteca de comunicação do *front-end*, conecta um usuário remetente e R usuários receptores a um mesmo projeto. A cada amostra, o remetente envia uma alteração, e o *script* registra o intervalo entre o envio e o recebimento do novo estado por cada receptor. Todos os usuários são executados no mesmo processo, de modo que os dois instantes são lidos do mesmo relógio. Em todas as amostras, a alteração é a renomeação de uma mesma tabela, o que mantém constante o tamanho das mensagens. Foram medidos grupos de 2, 5 e 10 usuários ($R = 1$, 4 e 9), com 100 amostras por grupo, precedidas de 5 amostras de aquecimento descartadas e separadas por intervalos de 200 milissegundos. Para cada grupo, foram calculados a média, o desvio padrão, a mediana, os percentis 95 e 99, o valor máximo e o número de mensagens perdidas.

A segunda é a latência percebida pelo usuário: o intervalo entre o clique no botão de adicionar tabela no navegador de um membro e o aparecimento da tabela no navegador de outro membro, medido pelo Playwright em 30 amostras. Diferentemente da primeira, essa medida inclui o intervalo de 1,5 segundo que antecede o envio automático das alterações e o tempo de desenho do diagrama.

Os valores obtidos foram comparados aos limites de tempo de resposta propostos por NIELSEN (1993): até 0,1 segundo, o usuário percebe a resposta como imediata; até 1 segundo, percebe o atraso, mas o seu fluxo de raciocínio não é interrompido. As medições foram feitas em dois ambientes. No ambiente local, todos os componentes do protótipo foram executados em contêineres Docker no PC descrito na Subseção 3.1.1, sem tráfego pela rede externa. No ambiente publicado, o protótipo foi executado na plataforma Render, no plano gratuito e na região da Virgínia, nos Estados Unidos, com as medições feitas a partir do mesmo PC, no Brasil.

3.2.10 Design da Interface do Usuário

O *design* da *interface* da aplicação foi elaborado para garantir uma experiência de usuário intuitiva e eficaz, combinando a familiaridade de ferramentas de modelagem de

dados estabelecidas com a usabilidade e a estética de aplicações *web* modernas.

A inspiração inicial para a estrutura geral da interface foi o MySQL Workbench. Esta escolha visou facilitar a transição para usuários já familiarizados com este ambiente de modelagem de dados. No entanto, a *interface* foi adaptada e enriquecida com padrões de *design* modernos comumente encontrados em aplicações *web*, tais como:

- Menu Lateral: Um menu lateral fixo oferece acesso rápido e fácil a todas as funcionalidades principais da aplicação.
- *Modals*: Janelas *modals* são utilizadas para apresentar informações adicionais ou solicitar confirmação do usuário em tarefas específicas, mantendo o contexto da tela principal.
- Elementos Visuais: Utilização de elementos visuais consistentes e modernos para guiar o usuário e melhorar a clareza da informação.

Um componente central da *interface* é a visualização e manipulação interativa dos diagramas. Para essa funcionalidade, foi integrada a biblioteca GoJS. Ela permite a criação de diagramas complexos e dinâmicos, com recursos como arrastar e soltar, *zoom*, e personalização visual, proporcionando uma experiência de modelagem de dados interativa. Essa abordagem torna a criação e visualização de diagramas uma parte central da experiência do usuário, facilitando a compreensão e manipulação das estruturas de dados.

A combinação do design conhecido do MySQL Workbench com elementos modernos de aplicações *web* e a biblioteca GoJS indicam que uma *interface* intuitiva e eficaz para a modelagem de dados e a colaboração em tempo real possa ser construída.

3.2.11 Idioma do Projeto

A língua inglesa exerce uma posição dominante na área da Tecnologia da Informação. Conforme (ALVES et al., 2022), a globalização e a competitividade do mercado tornaram o inglês praticamente nativo no setor tecnológico, sendo a língua predominante nas áreas de divulgação científica, comércio e relações internacionais. Essa realidade se manifesta de forma concreta na prática do desenvolvimento de *software*: os comandos e as palavras reservadas de todas as principais linguagens de programação são estruturados em inglês, assim como os métodos ágeis amplamente adotados pela indústria, como o Scrum e o Kanban.

Diante desse contexto, a adoção do inglês como idioma padrão para todos os artefatos técnicos produzidos neste projeto foi uma decisão deliberada, fundamentada nas boas práticas da Engenharia de *Software*. Esta escolha está em consonância com os

princípios do Código Limpo (*Clean Code*), que preconiza o uso de nomes significativos e autoexplicativos (MARTIN, 2009). A utilização do inglês garante que a nomenclatura empregada no código-fonte, nos esquemas de banco de dados e nos diagramas seja consistente com a terminologia das tecnologias, bibliotecas e *frameworks* adotados no projeto, evitando ambiguidades e facilitando a compreensão e a manutenção da base de código.

Esta padronização foi aplicada de forma abrangente nos seguintes artefatos do projeto:

- Código-fonte: nomes de classes, métodos, variáveis e atributos foram definidos em inglês tanto no *back-end* (Spring/Java) quanto no *front-end* (Angular/TypeScript), mantendo coerência com os próprios nomes das APIs e as convenções dos *frameworks* utilizados.
- Banco de dados: os nomes das tabelas, colunas e restrições no PostgreSQL, bem como os nomes das coleções e campos dos documentos no MongoDB, foram definidos em inglês. Essa decisão garante consistência com as anotações do Spring Data JPA e do Spring Data MongoDB, que realizam o mapeamento objeto-relacional e objeto-documento a partir dos identificadores declarados no código-fonte.
- Diagramas: os nomes de entidades, atributos e relacionamentos nos diagramas ER foram especificados em inglês, garantindo uniformidade com os demais artefatos técnicos do projeto.
- *Endpoints* da API REST: os caminhos dos recursos e as operações expostas pela API foram nomeados em inglês, seguindo as convenções estabelecidas pelo estilo arquitetural REST.
- Controle de versão: as mensagens de *commit* e os nomes de ramificações (*branches*) no repositório *git* foram redigidos em inglês, promovendo a rastreabilidade e a clareza do histórico de desenvolvimento.

A adoção uniforme do inglês como idioma técnico em todos os artefatos do projeto reduziu o atrito cognitivo entre as camadas da aplicação, facilitou a leitura e a manutenção do código e posicionou o trabalho em conformidade com os padrões internacionais da indústria de desenvolvimento de *software*.

4 Resultados

Esta seção apresenta os resultados obtidos ao longo do desenvolvimento do protótipo de modelador de diagramas ER colaborativo. Os resultados são organizados nas seguintes etapas: o diagrama de caso de uso, os modelos de dados, a investigação da biblioteca de diagramação, a arquitetura implementada e as funcionalidades do protótipo.

4.1 Diagrama de Caso de Uso

O diagrama de caso de uso detalha as interações entre os atores e o sistema. Foram identificados três atores distintos, cada um representando um perfil de acesso diferente dentro da plataforma: *Viewer*, *Editor* e *Owner*. Para preservar a legibilidade, os diagramas foram divididos em partes complementares, cada uma retomando o caso de uso que a conecta às demais. A especificação completa de cada caso de uso encontra-se no Anexo A.

O ator *Viewer*, apresentado na Figura 1, representa o colaborador com acesso somente leitura[1]. A listagem dos projetos dos quais faz parte inclui o *login*, de modo que nenhuma ação é realizada sem autenticação. A partir dessa listagem, o *Viewer* pode visualizar os diagramas associados aos projetos, caso de uso que estende a listagem.

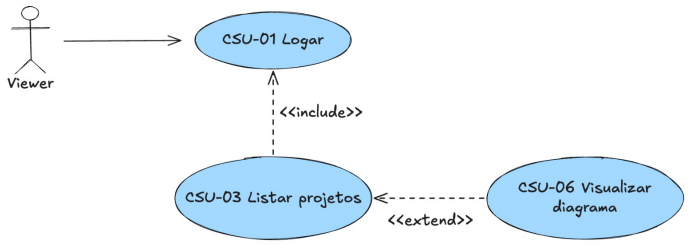


Figura 1 – Diagrama de caso de uso do ator Viewer

Fonte: Elaborado pelo autor

O ator *Editor* herda as capacidades do *Viewer* e adiciona a permissão de editar diagramas. Como o conjunto de casos de uso desse ator é extenso, o diagrama foi dividido

[1] O diagrama completo do ator *Viewer* pode ser acessado em: <https://excalidraw.com/#json=3hPCWIRcAVBBXBanascky,2XErJH4B4bmNwDk8wUHDSQ>.

em cinco partes[2], apresentadas nas Figuras 2 a 6. A Figura 2 apresenta o fluxo principal: a listagem de projetos inclui o *login* e é estendida pela visualização do diagrama. A visualização, por sua vez, é estendida pela exportação do *script* SQL e pela edição do diagrama, caso de uso que concentra as operações de modelagem detalhadas nas partes seguintes.

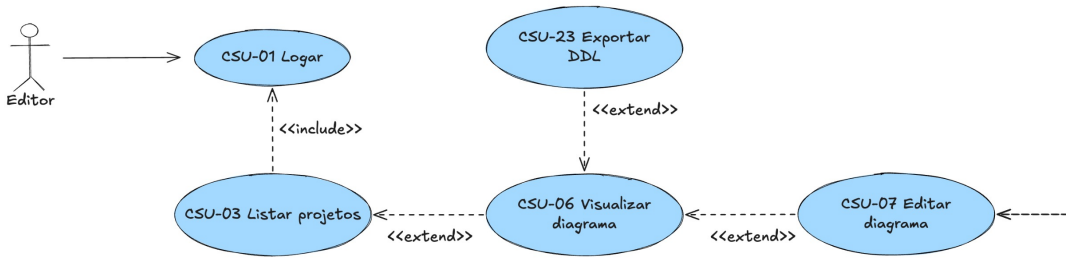


Figura 2 – Diagrama de caso de uso do ator Editor destacando o fluxo principal

Fonte: Elaborado pelo autor

A Figura 3 detalha as operações sobre tabelas que estendem o CSU-07: adicionar, editar e remover. A edição de uma tabela sempre inclui a listagem dos seus atributos e pode ser estendida pela adição de um novo atributo.

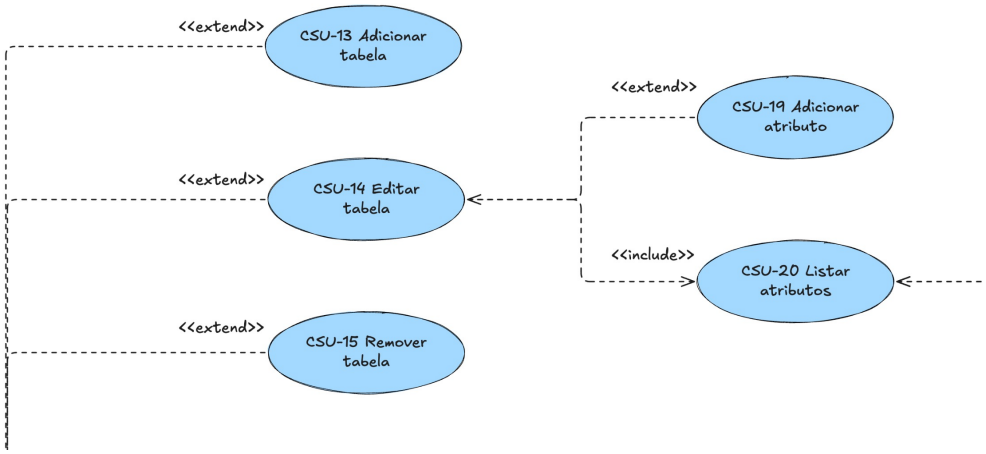


Figura 3 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de tabelas

Fonte: Elaborado pelo autor

[2] O diagrama completo do ator *Editor* pode ser acessado em: <https://excalidraw.com/#json=vpDsVhuufUh1xut1jUWfm,LWblOOztfl4Bw4qZTetTzg>.

A Figura 4 dá continuidade ao CSU-20 da parte anterior, apresentando a edição e a remoção de atributos como extensões da listagem.

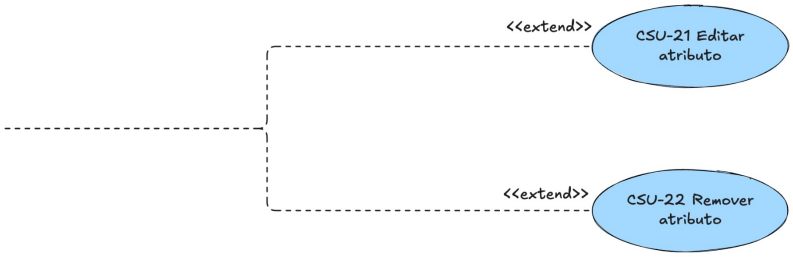


Figura 4 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de atributos

Fonte: Elaborado pelo autor

A Figura 5 retoma o CSU-07 e apresenta outras extensões da edição do diagrama: a adição e a remoção de relacionamentos entre entidades e o reposicionamento de elementos na área de desenho.

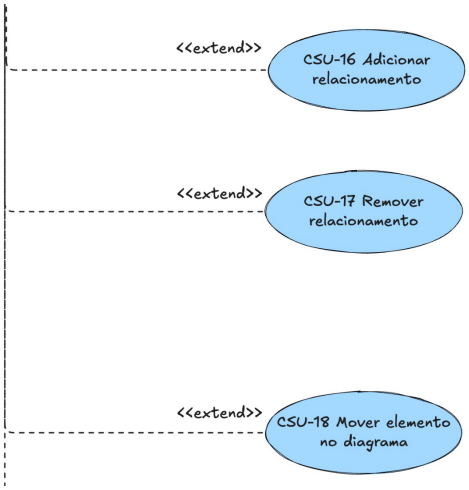


Figura 5 – Diagrama de caso de uso do ator Editor destacando o gerenciamento de relacionamentos e a movimentação de elementos

Fonte: Elaborado pelo autor

Por fim, a Figura 6 apresenta a importação do *script* SQL, que também estende o CSU-07.

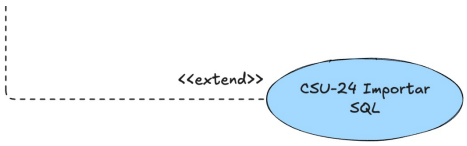


Figura 6 – Diagrama de caso de uso do ator Editor destacando a importação SQL

Fonte: Elaborado pelo autor

O ator *Owner* possui o conjunto mais amplo de permissões: além de todos os casos de uso do *Editor*, é o responsável pelo ciclo de vida dos projetos e pela gestão do time de colaboradores. Por reunir todas as funcionalidades do sistema, o seu diagrama foi dividido em sete partes[3], apresentadas nas Figuras 7 a 13. A Figura 7 mostra o ponto de partida: tanto a criação quanto a listagem de projetos incluem o *login*, de modo que nenhuma ação é realizada sem autenticação.

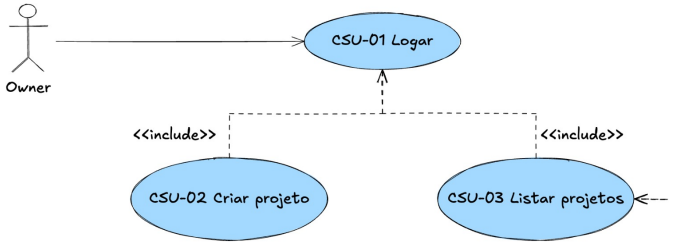


Figura 7 – Diagrama de caso de uso do ator Owner destacando o *login* e a criação e listagem de projetos

Fonte: Elaborado pelo autor

[3] O diagrama completo do ator *Owner* pode ser acessado em: <https://excalidraw.com/#json=4QSsiWj6YhGr5VoOFNs1V,EjxswZkUFHg9A2g9XntIOw>.

A Figura 8 apresenta as extensões da listagem de projetos voltadas à sua gestão: a remoção e a edição do projeto. A edição sempre inclui a listagem dos membros do time e pode ser estendida pela adição de novos membros. A listagem de membros, por sua vez, é estendida pela edição e pela remoção de um membro.

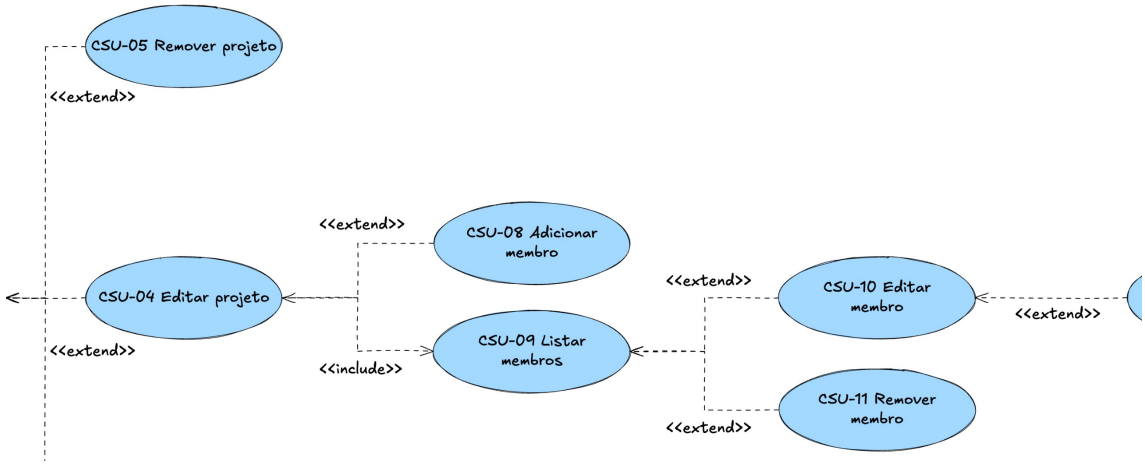


Figura 8 – Diagrama de caso de uso do ator Owner destacando a gestão do projeto e do time

Fonte: Elaborado pelo autor

A Figura 9 dá continuidade ao CSU-09 e mostra que a edição de um membro é estendida pela alteração do seu papel de acesso no projeto.

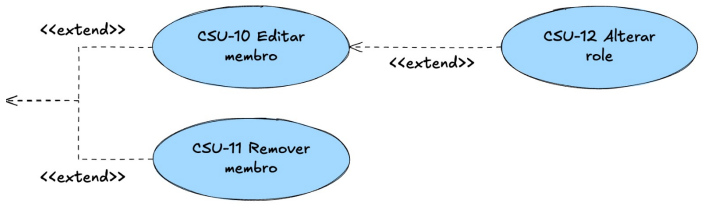


Figura 9 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de membros

Fonte: Elaborado pelo autor

A partir da Figura 10, o diagrama percorre as operações de modelagem, que são as

mesmas do *Editor*. A visualização do diagrama estende a listagem de projetos e é estendida pela exportação do *script* SQL e pela edição do diagrama.

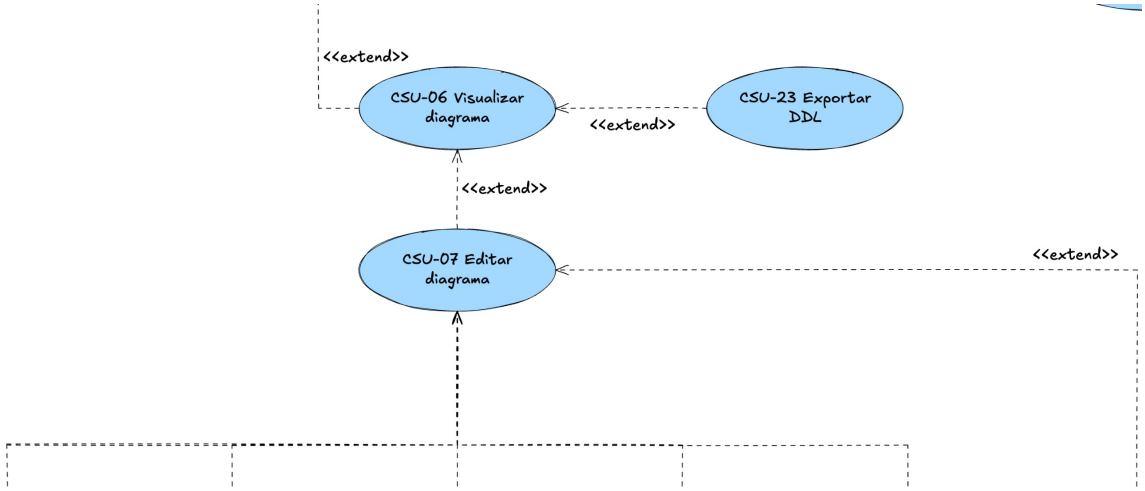


Figura 10 – Diagrama de caso de uso do ator Owner destacando a visualização e a edição do diagrama

Fonte: Elaborado pelo autor

A Figura 11 apresenta as extensões do CSU-07 relativas às tabelas (adicionar, editar e remover) e a adição de relacionamentos. A edição de uma tabela sempre inclui a listagem dos seus atributos e pode ser estendida pela adição de um novo atributo.

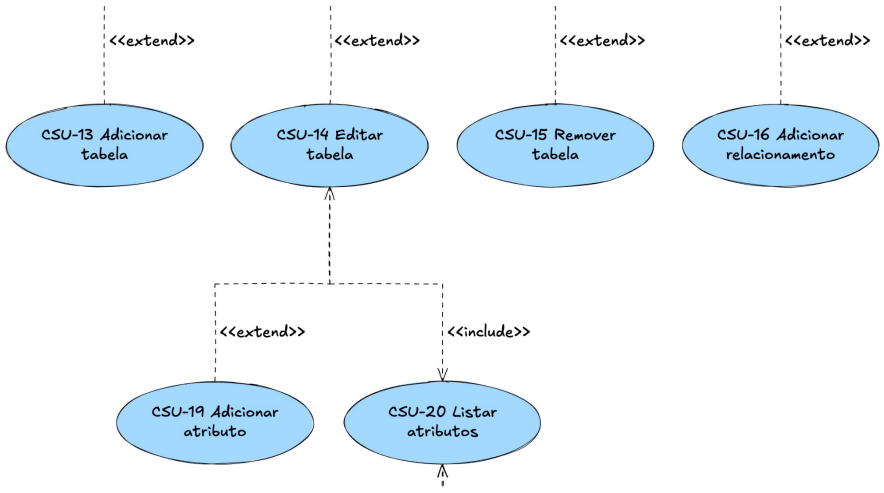


Figura 11 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de tabelas

Fonte: Elaborado pelo autor

A Figura 12 dá continuidade ao CSU-20, apresentando a edição e a remoção de atributos como extensões da listagem.

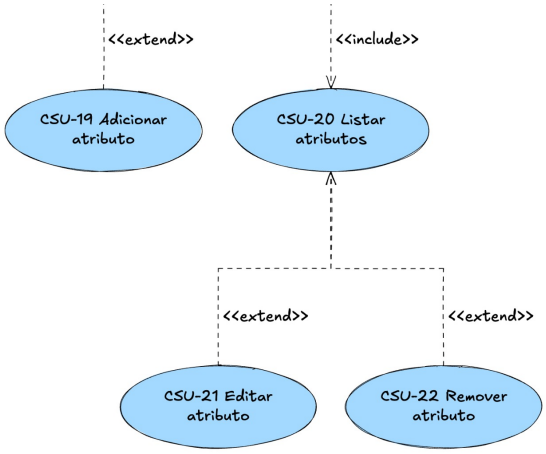


Figura 12 – Diagrama de caso de uso do ator Owner destacando o gerenciamento de atributos

Fonte: Elaborado pelo autor

Por fim, a Figura 13 reúne as demais extensões do CSU-07: a remoção de relacionamentos, o reposicionamento de elementos na área de desenho e a importação do *script* SQL.

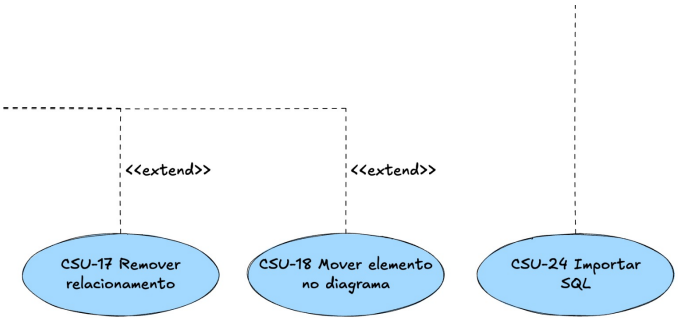


Figura 13 – Diagrama de caso de uso do ator Owner destacando relacionamentos, movimentação de elementos e importação SQL

Fonte: Elaborado pelo autor

4.2 Modelos de Dados

As figuras 14, 15 e 16 apresentam, respectivamente, os artefatos de modelagem de dados produzidos no projeto: o modelo conceitual e o modelo lógico do banco de dados

relacional, e o esquema não-relacional com MongoDB.

O modelo conceitual define as entidades e relacionamentos de alto nível, representando a estrutura geral dos dados. Ao todo, sete entidades compõem o modelo (*users*, *teams*, *projects*, *diagrams*, *entities*, *attributes* e *relationships*) conectadas por relações que refletem a hierarquia usuário-time-projeto-diagrama e a composição interna de cada diagrama em entidades, atributos e relacionamentos.

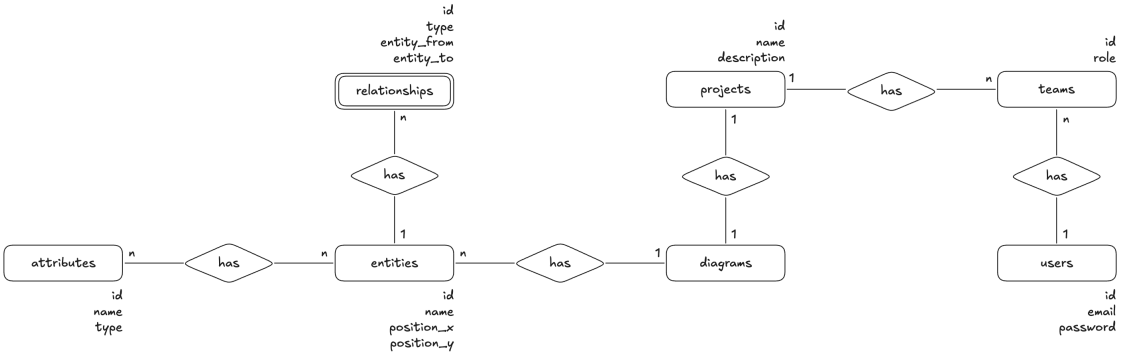


Figura 14 – Modelo conceitual do banco de dados

Fonte: Elaborado pelo autor

O modelo lógico detalha a estrutura das tabelas e seus atributos, considerando as características do sistema de gerenciamento de banco de dados utilizado. O esquema relacional é composto por quatro tabelas (*teams*, *projects*, *users* e *refresh_tokens*) relacionadas por chaves estrangeiras que garantem a integridade referencial entre usuários, times e projetos, além do controle de sessão por meio dos *tokens* de renovação.

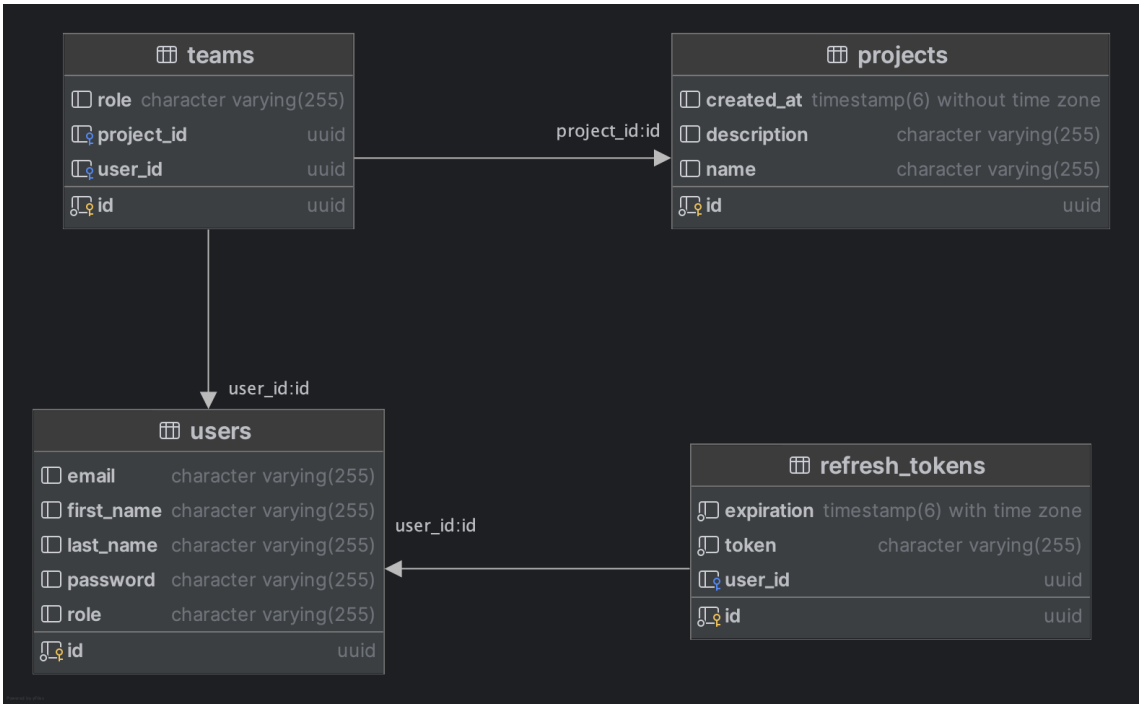


Figura 15 – Modelo lógico do banco de dados

Fonte: Elaborado pelo autor

O esquema documental projetado para o MongoDB contempla as coleções, os campos e os tipos de dados. O esquema é composto por uma única coleção chamada *diagram*, com cinco campos: o identificador, o discriminador de classe, a referência ao projeto e os campos *linkData* e *nodeData*, detalhados a seguir, que armazenam, respectivamente, os relacionamentos e as entidades do diagrama em formato serializado.

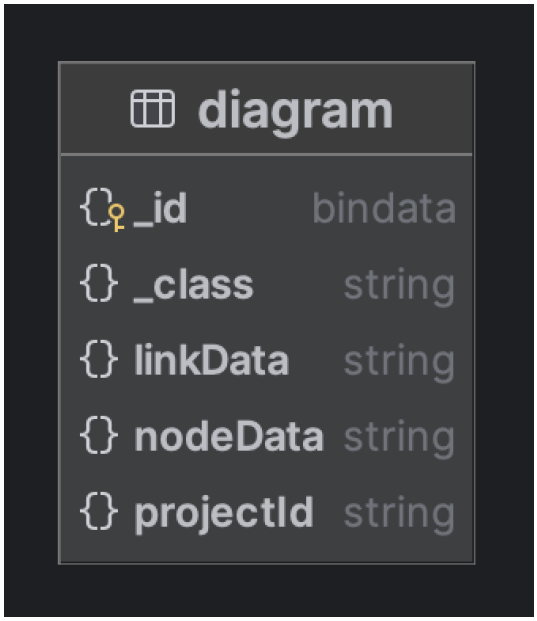


Figura 16 – Esquema do banco de dados não-relacional

Fonte: Elaborado pelo autor

Os campos *linkData* e *nodeData* possuem, na devida ordem, os seguintes formatos:

```
{
    "from": string,
    "to": string,
    "text": string,
    "toText": integer,
    "fromColumn": string
}
```

Quadro 1 – Estrutura de dados do campo *linkData*

Fonte: Elaborado pelo autor

```
{
    "id": string,
    "key": string,
    "items": [
        {
        "name": string,
        "type": string,
        "pk": boolean,
        "fk": boolean,
        "unique": boolean,
        "notNull": boolean,
        "autoIncrement": boolean,
        "defaultValue": string
        }
    ],
    "location": {
        "x": string,
        "y": string
    }
}
```

Quadro 2 – Estrutura de dados do campo *nodeData*

Fonte: Elaborado pelo autor

4.3 Investigação da Biblioteca de Diagramação

A área de desenho concentra a representação visual das entidades, dos atributos e dos relacionamentos, bem como a interação do usuário com o diagrama. Para a sua construção, foram investigadas, na fase inicial do projeto, três alternativas: a biblioteca GoJS, a biblioteca JointJS e o desenvolvimento de elementos gráficos próprios diretamente sobre os recursos de desenho do navegador. As alternativas foram avaliadas segundo cinco critérios: a disponibilidade de recursos para a representação de diagramas ER, a disponibilidade de recursos de interação com o diagrama, a forma de representação dos dados do diagrama, a integração com o Angular e as condições de licenciamento.

O desenvolvimento de elementos próprios ofereceria controle total sobre a aparência e o comportamento do diagrama, além de dispensar qualquer custo de licença. Em contrapartida, exigiria implementar recursos que as bibliotecas de diagramação já oferecem, como a seleção de elementos, o arrastar e soltar, o *zoom*, o traçado das linhas de relacionamento entre as entidades e o desfazer e refazer de ações. Conforme a revisão de estudos industriais conduzida por MOHAGHEGHI e CONRADI (2007), o reuso sistemático de componentes de *software* está associado a uma menor densidade de defeitos e a ganhos de produtividade.

Considerando que o protótipo foi desenvolvido por um único desenvolvedor e dentro do prazo de um trabalho de conclusão de curso, essa alternativa foi descartada, de modo que o esforço do projeto pudesse ser direcionado à colaboração em tempo real, foco do objetivo geral deste trabalho.

A JointJS é uma biblioteca de diagramação baseada em SVG, cujo núcleo é distribuído sob licença de código aberto, enquanto recursos avançados de *interface* são oferecidos em uma versão comercial denominada JointJS+ (Client IO, 2026b). O exemplo de diagrama ER disponibilizado pelo fabricante pertence a essa versão comercial (Client IO, 2026a). A biblioteca já foi empregada em ferramentas acadêmicas de modelagem colaborativa: ZWEIHOFF, NAUJOKAT e STEFFEN (2019) a utilizaram como camada de visualização do Pyro, um ambiente de modelagem colaborativa executado no navegador, no qual os elementos gráficos são manipulados de forma interativa. Por ser baseado em SVG, cada elemento do diagrama torna-se parte da estrutura da página, o que permite a sua inspeção pelas ferramentas do navegador e a sua leitura por tecnologias assistivas (IVANČEVIĆ; JEŽIĆ; LOKNAR, 2026).

A GoJS, por sua vez, é uma biblioteca comercial que realiza a renderização sobre o elemento Canvas do HTML. A investigação identificou um conjunto de capacidades diretamente aplicáveis ao modelador proposto. A biblioteca permite definir modelos visuais reutilizáveis para as entidades e para os relacionamentos, cuja aparência é vinculada automaticamente aos dados, de modo que uma alteração em um atributo é refletida no diagrama sem necessidade de redesenho manual. Ela dispõe ainda de um painel em formato de tabela, adequado à representação de uma entidade com a sua lista de atributos, e o fabricante disponibiliza um exemplo oficial de diagrama ER (Northwoods Software Corporation, 2026b). Arrastar e soltar, *zoom*, seleção, desfazer e refazer e o traçado das linhas de relacionamento são oferecidos pela própria biblioteca, e o fabricante mantém documentação específica para a integração com o Angular (Northwoods Software Corporation, 2024).

Dentre essas capacidades, a forma de representação dos dados teve papel central na escolha. A GoJS separa o modelo do diagrama da sua apresentação visual e serializa esse modelo em JSON, na forma de uma lista de nós e de uma lista de ligações. Essa estrutura foi aproveitada diretamente no esquema documental do MongoDB, nos campos *linkData* e *nodeData* apresentados nos Quadros 1 e 2, sem a necessidade de conversões entre formatos. Além disso, as alterações no modelo são agrupadas em transações e notificadas pela biblioteca, o que permitiu capturar cada modificação realizada por um colaborador e propagá-la aos demais por meio da conexão WebSocket.

A investigação também evidenciou limitações da GoJS. A primeira é o licenciamento: a biblioteca não é de código aberto e, sem uma chave de licença, exibe uma marca d'água sobre o diagrama, havendo uma modalidade gratuita para fins acadêmicos e não comerciais

(Northwoods Software Corporation, 2026a). Essa condição atende ao escopo deste trabalho, mas representa um custo para uma eventual adoção comercial, conforme discutido nas limitações do protótipo. A segunda decorre da renderização em Canvas: o conteúdo desenhado não fica disponível como elementos da página, de modo que não pode ser lido por tecnologias assistivas sem tratamento adicional (IVANČEVIĆ; JEŽIĆ; LOKNAR, 2026) nem consultado pelos testes automatizados da mesma forma que os demais elementos da *interface*. Essa característica motivou o isolamento da renderização em componentes próprios, descrito na seção de testes, para que o restante da aplicação pudesse ser testado de forma independente. A terceira refere-se à integração com o mecanismo de detecção de mudanças do Angular, que exigiu cuidado adicional, conforme discutido no capítulo seguinte.

A Tabela 4 sintetiza a comparação entre as três alternativas investigadas.

Tabela 4 – Comparação entre as alternativas investigadas para a área de desenho

Critério	**GoJS**	**JointJS**	**Elementos próprios**
Exemplo oficial de diagrama ER	Sim	Somente na versão comercial	Não se aplica
Seleção, arrastar e soltar e *zoom*	Oferecidos pela biblioteca	Oferecidos pela biblioteca	A implementar
Modelo de dados em JSON	Sim	Sim	A implementar
Integração com o Angular	Documentada pelo fabricante	Documentada pelo fabricante	A implementar
Licença	Comercial, com uso acadêmico gratuito	Núcleo aberto e versão comercial	Sem custo

Fonte: Elaborado pelo autor

Diante desses resultados, a GoJS foi adotada como biblioteca de diagramação do protótipo. A decisão se justifica por ela reunir, sem custo no contexto acadêmico, os recursos utilizados na modelagem ER do protótipo e por oferecer um modelo de dados que se integrou diretamente tanto ao esquema de persistência quanto ao mecanismo de colaboração em tempo real. As limitações identificadas, especialmente o licenciamento comercial, foram consideradas aceitáveis para um protótipo e são retomadas na discussão dos resultados como pontos de atenção para trabalhos futuros.

4.4 Arquitetura do Protótipo

O protótipo foi construído segundo a arquitetura cliente-servidor, composta por dois módulos independentes: o *back-end*, implementado com Spring Boot, e o *front-end*, desenvolvido com Angular. A Figura 17 apresenta uma visão geral dos componentes do sistema e dos canais de comunicação entre eles.

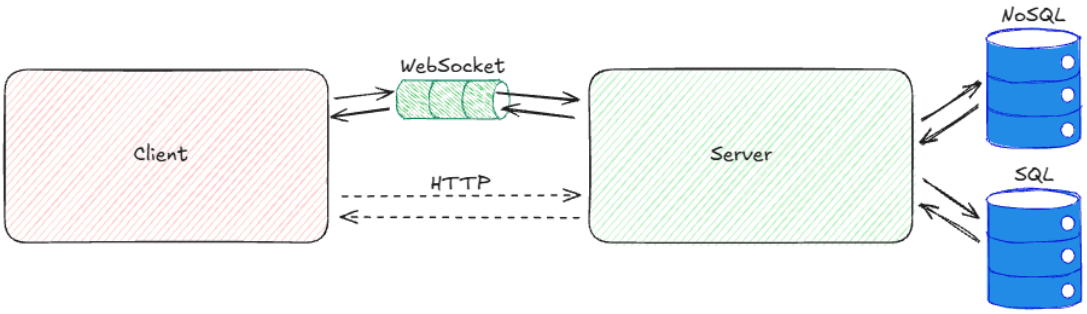


Figura 17 – Arquitetura do protótipo

Fonte: Elaborado pelo autor

A comunicação entre os módulos ocorre por dois canais distintos: requisições HTTP para as operações de persistência e autenticação, e uma conexão persistente via WebSocket para a sincronização colaborativa em tempo real.

O *back-end* foi organizado em camadas seguindo o padrão MVC, separando as responsabilidades de controle, regras de negócio e acesso a dados. A segurança da API é gerenciada por meio de *tokens* JWT armazenados em *cookies* no navegador do cliente, protegendo os recursos contra acesso não autorizado. A comunicação em tempo real é viabilizada por um servidor WebSocket dedicado, responsável por receber as ações dos colaboradores e propagá-las a todos os usuários conectados ao mesmo projeto. Essa escolha evita a sobrecarga de requisições HTTP repetidas (*polling*) para manter os colaboradores sincronizados, reduzindo a latência percebida durante a edição simultânea do diagrama.

O *front-end* é estruturado como uma SPA, com responsabilidades distribuídas entre componentes de interface, serviços de dados e interceptadores de requisição. O componente central do editor integra a biblioteca GoJS para a renderização e manipulação interativa do diagrama no *canvas*, enquanto serviços dedicados encapsulam a comunicação em tempo real e as operações de importação e exportação SQL.

4.5 Funcionalidades Implementadas

As subseções a seguir descrevem as funcionalidades que compõem o protótipo desenvolvido, cada uma concebida para atender diretamente às necessidades dos perfis *Viewer*, *Editor* e *Owner* apresentados anteriormente.

4.5.1 Autenticação e Autorização

O protótipo implementa um fluxo de autenticação com credenciais de acesso. Após a autenticação bem-sucedida, o servidor emite dois *tokens* de segurança: um de curta duração para as requisições e outro de renovação, ambos armazenados de forma segura no navegador do cliente. As rotas da aplicação são protegidas tanto no lado do cliente quanto no servidor, garantindo que apenas usuários autenticados acessem os recursos. Quando o *token* de acesso expira, a renovação ocorre de forma automática e transparente ao usuário, sem necessidade de novo *login*.

A Figura 18 ilustra a arquitetura de autenticação e autorização implementada no *back-end*. Toda requisição originada pelo cliente passa pelo filtro JWT, que valida o *token* presente no *cookie*; caso inválido, a requisição é rejeitada com o código 401 (*Unauthorized*). Requisições válidas avançam para duas camadas de autorização em série: a primeira verifica permissões com base na URL acessada e a segunda aplica restrições mais granulares ao nível do método, controlando o acesso de acordo com o perfil do usuário autenticado. O descumprimento de qualquer uma dessas camadas resulta em resposta 403 (*Forbidden*); somente após a aprovação em ambas a requisição alcança os controladores da aplicação.

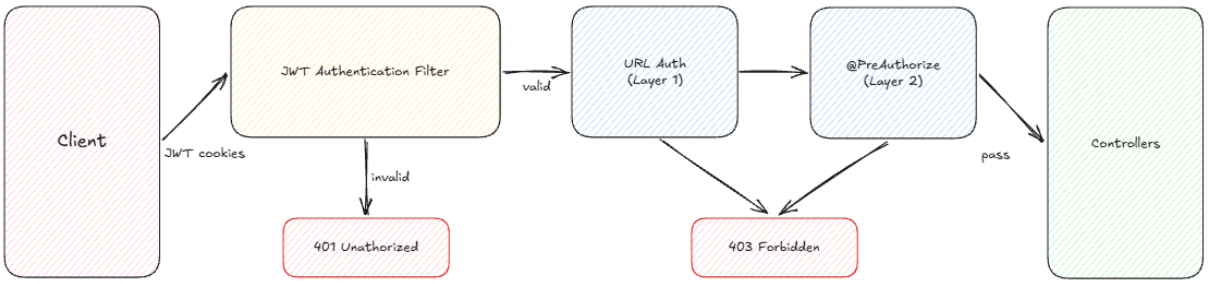


Figura 18 – Arquitetura de autenticação e autorização do *back-end*

Fonte: Elaborado pelo autor

4.5.2 Gerenciamento de Projetos e Times

O sistema permite ao usuário criar e gerenciar projetos de modelagem. Cada projeto possui um time de colaboradores, para o qual o criador pode convidar outros usuários cadastrados na plataforma. A gestão de projetos e times é realizada por meio

de uma interface de *dashboard*, com operações de criação, leitura, atualização e exclusão comunicadas ao servidor e persistidas no banco de dados.

4.5.3 Editor de Diagramas ER

O editor de diagramas constitui o componente central do protótipo. Construído com a biblioteca GoJS, o *canvas* permite adicionar e remover entidades (tabelas), gerenciar seus atributos com configuração de tipo de dado, chave primária, chave estrangeira, unicidade, restrição não nulo, auto-incremento e valor padrão, criar e excluir relacionamentos entre entidades com definição de cardinalidade, além de reposicionar livremente os elementos na área de desenho. O estado completo do diagrama (entidades, atributos, relacionamentos e posicionamentos) é persistido na base de dados ao final de cada sequência de ações.

4.5.4 Colaboração em Tempo Real

A funcionalidade de colaboração permite que múltiplos usuários de um mesmo time trabalhem simultaneamente no mesmo diagrama. Ao abrir um projeto, o cliente estabelece uma conexão persistente com o servidor, autenticada pelo *token* JWT, e passa a receber as alterações dos demais membros. O servidor só aceita a conexão de usuários autenticados e só entrega as alterações de um projeto aos seus membros. As alterações de cada usuário são enviadas 1,5 segundo depois da última edição, contendo apenas os elementos alterados, e o servidor as combina com as dos demais antes de distribuí-las, conforme a estratégia descrita na Metodologia. Quando um usuário abre uma tabela para edição, os demais colaboradores são notificados do bloqueio e ficam impedidos de editá-la até que ele encerre a edição, que a sua conexão termine ou que se passem 5 minutos.

4.5.5 Importação e Exportação de Diagramas

O protótipo suporta a interoperabilidade com outros ambientes por meio de operações SQL no dialeto MySQL. A exportação converte o modelo do diagrama em instruções SQL de criação de tabelas, enquanto a importação analisa um *script* SQL fornecido pelo usuário para reconstruir o diagrama correspondente no *canvas*. Ambas as operações são processadas pelo *back-end* e o resultado é retornado ao usuário pela interface da aplicação.

4.6 Testes e Validação do Protótipo

Esta seção apresenta os resultados da estratégia de testes descrita na Subseção 3.2.9: a composição dos testes automatizados e a cobertura de código, o teste de mutação, os testes de sistema e as medições de desempenho. Além dos testes automatizados, as APIs REST do *back-end* foram testadas manualmente com o Postman, nos fluxos de autenticação

e de renovação de *token*, de gestão de projetos e times e de gravação do diagrama, e a interface foi verificada manualmente nos navegadores Google Chrome e Mozilla Firefox.

4.6.1 Testes Automatizados e Cobertura de Código

A Tabela 5 apresenta a quantidade de testes automatizados de cada nível. Todos foram aprovados.

Tabela 5 – Testes automatizados por nível

Nível	**Projeto**	**Ferramentas**	**Testes**
Unitário	*Back-end*	JUnit e Mockito	350[a]
Integração	*Back-end*	Spring Boot e H2	5
Unitário	*Front-end*	Jasmine e Karma	636
Sistema	Protótipo completo	Playwright	7
Total			**998**

[a] Inclui 11 testes que verificam as regras de acesso com a aplicação carregada.
Fonte: Elaborado pelo autor

A medição de cobertura atingiu 100% em todos os indicadores nos dois projetos. A Tabela 6 apresenta o relatório do *back-end*, por pacote.

Tabela 6 – Relatório de cobertura de código dos testes do *back-end*

Pacote	**Instruções**	**Ramos**	**Linhas**	**Métodos**	**Classes**
com.erd.core.service	100%	100%	719/719	135/135	12/12
com.erd.core.config	100%	100%	131/131	31/31	8/8
com.erd.core.controller	100%	100%	136/136	36/36	7/7
com.erd.core.dto.response	100%	n/a	130/130	71/71	7/7
com.erd.core.dto.request	100%	n/a	101/101	68/68	11/11
com.erd.core.model	100%	100%	90/90	53/53	4/4
com.erd.core.dto	100%	n/a	92/92	60/60	6/6
com.erd.core.filter	100%	100%	36/36	7/7	2/2
com.erd.core.mapper	100%	n/a	28/28	11/11	4/4
com.erd.core.dto.collaboration	100%	n/a	27/27	14/14	1/1
com.erd.core.model.mongo	100%	n/a	19/19	9/9	1/1
com.erd.core.enumeration	100%	n/a	4/4	2/2	2/2
com.erd.core.dto.error	100%	n/a	11/11	6/6	1/1
com.erd.core.advice	100%	n/a	2/2	2/2	1/1
com.erd.core.exception	100%	n/a	2/2	1/1	1/1
com.erd.core	100%	n/a	3/3	2/2	1/1
Total	**5.825/5.825**	**278/278**	**1.531/1.531**	**508/508**	**69/69**

Fonte: Elaborado pelo autor

A Tabela 7 apresenta o relatório equivalente do *front-end*, por diretório.

Tabela 7 – Relatório de cobertura de código dos testes do *front-end*

Diretório	**Instruções**		**Ramos**		**Funções**		**Linhas**	
	Cob.	**Qtd.**	**Cob.**	**Qtd.**	**Cob.**	**Qtd.**	**Cob.**	**Qtd.**
app	100%	23/23	100%	2/2	100%	10/10	100%	21/21
app/component/entity-edit-form	100%	55/55	100%	33/33	100%	15/15	100%	54/54
app/component/login	100%	18/18	100%	1/1	100%	5/5	100%	16/16
app/component/modal/project	100%	194/194	100%	76/76	100%	48/48	100%	191/191
app/component/register	100%	22/22	100%	3/3	100%	5/5	100%	21/21
app/component/side-bar	100%	95/95	100%	25/25	100%	33/33	100%	94/94
app/component/table-editor	100%	30/30	100%	6/6	100%	9/9	100%	28/28
app/diagram	100%	641/641	100%	228/228	100%	176/176	100%	592/592
app/guards	100%	27/27	100%	9/9	100%	7/7	100%	25/25
app/interceptor	100%	47/47	100%	15/15	100%	11/11	100%	41/41
app/model/enum	100%	13/13	100%	2/2	100%	1/1	100%	13/13
app/service	100%	314/314	100%	112/112	100%	102/102	100%	295/295
app/shared	100%	8/8	100%	0/0	100%	6/6	100%	7/7
environments	100%	1/1	100%	0/0	100%	0/0	100%	1/1
Total	**100%**	**1488/1488**	**100%**	**512/512**	**100%**	**428/428**	**100%**	**1399/1399**

Fonte: Elaborado pelo autor

4.6.2 Teste de Mutação

A Tabela 8 apresenta o resultado do teste de mutação nos dois projetos.

Tabela 8 – Resultado do teste de mutação

Projeto	**Mutantes avaliados**	**Mortos**	**Sobreviventes**	**Escore**
Back-end	534	499	35	93,4%
Front-end	2.793[a]	2.253	540[b]	80,7%

[a] Dos 2.876 mutantes gerados, foram desconsiderados 49 que impediram a execução dos testes e 34 em trechos executados apenas no carregamento da aplicação, excluídos por configuração.
[b] Inclui 2 mutantes em trechos que nenhum teste executa.
Fonte: Elaborado pelo autor

No *back-end*, os 35 mutantes sobreviventes se distribuem em quatro grupos: 14 removem chamadas que preenchem campos de objetos, como as que registram a unicidade, o valor padrão e a posição das tabelas na importação de *scripts* SQL; 15 trocam o valor de retorno de um método por um valor vazio, nulo ou constante, entre eles dois que fazem verificações de permissão retornarem sempre verdadeiro; 5 invertem ou deslocam uma condição, entre elas a que define o instante de expiração do *token* de renovação; e 1 altera um valor numérico usado no posicionamento das tabelas importadas.

No *front-end*, os 540 mutantes não detectados concentram-se no componente do editor de diagramas (302) e no serviço que valida os *scripts* SQL importados (72). Por tipo, 221 alteram o conteúdo de textos, dos quais 126 são mensagens escritas no console do

navegador e os demais são valores como as cores associadas aos tipos de dados, os valores padrão dos atributos e os nomes usados para localizar chaves estrangeiras; 91 substituem uma condição por um valor fixo; 49 trocam chamadas de métodos sobre listas e textos; e os 179 restantes se distribuem entre outros 12 tipos de alteração.

4.6.3 Testes de Sistema

A Tabela 9 apresenta o resultado do roteiro de testes de sistema (Tabela 3) nos ambientes local e publicado. Os sete casos foram aprovados nas três execuções em cada ambiente.

Tabela 9 – Resultado dos testes de sistema

Caso	**Ambiente local**	**Ambiente publicado**
CT-01	Aprovado (3/3)	Aprovado (3/3)
CT-02	Aprovado (3/3)	Aprovado (3/3)
CT-03	Aprovado (3/3)	Aprovado (3/3)
CT-04	Aprovado (3/3)	Aprovado (3/3)
CT-05	Aprovado (3/3)	Aprovado (3/3)
CT-06	Aprovado (3/3)	Aprovado (3/3)
CT-07	Aprovado (3/3)	Aprovado (3/3)

Fonte: Elaborado pelo autor

No caso CT-07, o tempo entre o encerramento da conexão do *Editor* e a liberação da tabela para o *Owner* foi registrado. No ambiente local, foi de 0,07 a 0,17 segundo em 6 medições. No ambiente publicado, as 9 medições se dividiram em dois grupos: 3 entre 4,5 e 6,6 segundos e 6 entre 31,0 e 31,1 segundos.

4.6.4 Desempenho da Colaboração em Tempo Real

A Tabela 10 apresenta a latência de propagação medida no nível do protocolo, em milissegundos. Nenhuma das 1.400 entregas esperadas em cada ambiente foi perdida.

Tabela 10 – Latência de propagação no nível do protocolo (ms)

Ambiente	Usuários	Entregas	Média	Desvio padrão	Mediana	P95	P99	Máximo	Perdas
Local	2	100	8,1	0,7	7,9	9,4	10,8	11,1	0
Local	5	400	7,8	0,5	7,7	8,8	9,2	10,2	0
Local	10	900	7,9	1,1	7,7	8,8	14,9	15,2	0
Publicado	2	100	187,2	5,6	185,3	196,3	200,6	221,7	0
Publicado	5	400	178,6	19,6	178,9	192,3	251,9	481,9	0
Publicado	10	900	168,3	13,5	169,2	180,2	191,0	468,2	0

Fonte: Elaborado pelo autor

Como referência para o ambiente publicado, o tempo de ida e volta de uma requisição simples ao servidor, pelo mesmo caminho, teve mediana de 204 milissegundos em 20 requisições. A Tabela 11 apresenta a latência percebida pelo usuário, do clique em um navegador ao aparecimento da tabela no outro.

Tabela 11 – Latência percebida pelo usuário (ms)

Ambiente	Amostras	Média	Desvio padrão	Mediana	P95	Mínimo	Máximo
Local	30	1.532	9	1.530	1.547	1.517	1.551
Publicado	30	1.707	10	1.706	1.720	1.691	1.722

Fonte: Elaborado pelo autor

4.7 *Interface* do Usuário

Esta seção apresenta as telas que compõem a *interface* do protótipo, organizadas pelo fluxo de uso da aplicação.

4.7.1 Autenticação

A Figura 19 apresenta a tela de entrada da aplicação. O usuário informa seu *e-mail* e senha para autenticar-se, com a opção de manter a sessão ativa. O cadastro de novos usuários não está disponível pela interface gráfica, sendo realizado exclusivamente por meio da API do *back-end*. O Quadro 3 apresenta o comando utilizado para esse cadastro.

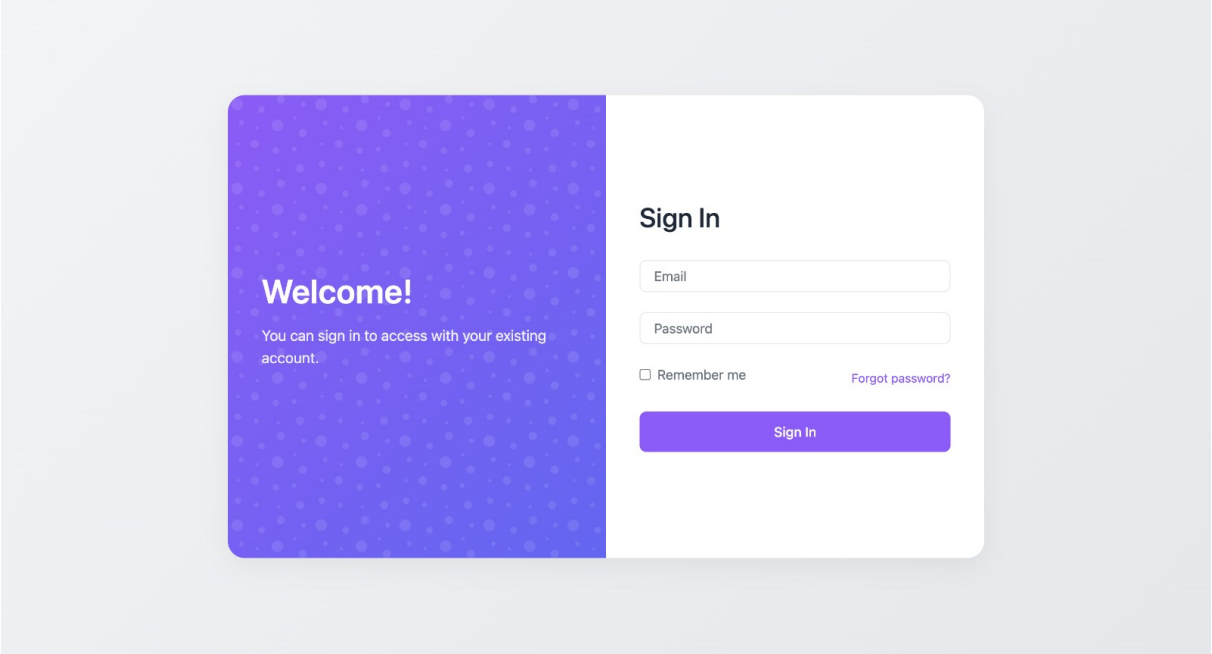


Figura 19 – Tela de *login*

Fonte: Elaborado pelo autor

```
curl -X POST https://erd-core.onrender.com/api/user \
    -H "Content-Type: application/json" \
    -d '{
      "firstName": "Test",
      "lastName": "Name",
      "email": "test.name@email.com",
      "password": "yourPassword123!",
      "role": "USER"
    }'
```

Quadro 3 – Comando para cadastro de usuário via API

Fonte: Elaborado pelo autor

4.7.2 Gerenciamento de Projetos e Times

Após o *login*, o usuário é direcionado à tela principal da aplicação, apresentada na Figura 20. O menu lateral lista os projetos dos quais o usuário faz parte e disponibiliza o botão para criação de um novo projeto. Na parte inferior do menu, são exibidas as informações do usuário autenticado e a opção de encerrar a sessão.



Figura 20 – Tela principal com listagem de projetos

Fonte: Elaborado pelo autor

Ao acionar o botão de criação, é exibido o *modal* apresentado na Figura 21. O usuário informa o nome e a descrição do projeto e confirma a operação. O projeto criado passa a aparecer imediatamente no menu lateral.

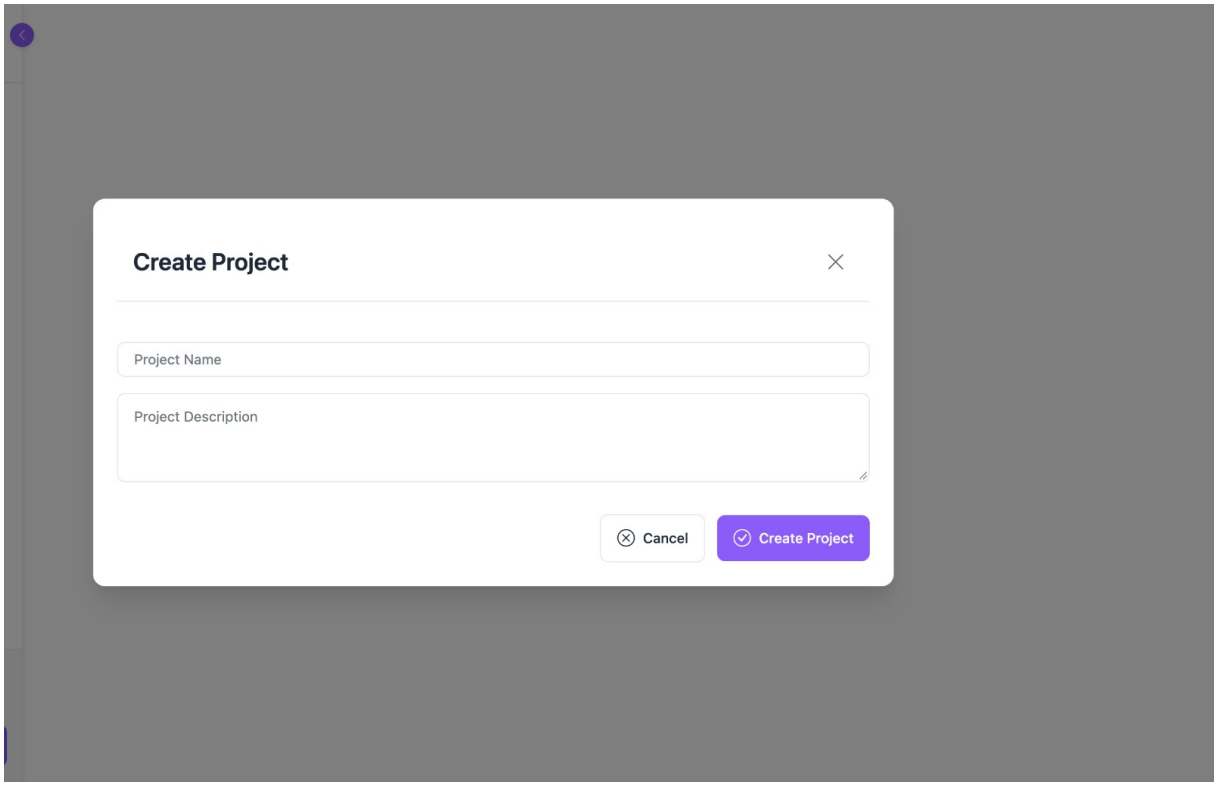


Figura 21 – *Modal* de criação de projeto

Fonte: Elaborado pelo autor

A Figura 22 ilustra o *modal* de edição de projeto, que reúne em uma única tela a edição dos dados do projeto e o gerenciamento do time de colaboradores. A seção *Team Members* lista os membros com seus respectivos papéis de acesso, e o *Owner* pode adicionar ou remover colaboradores e alterar seus papéis. A tela também disponibiliza a opção de excluir o projeto.

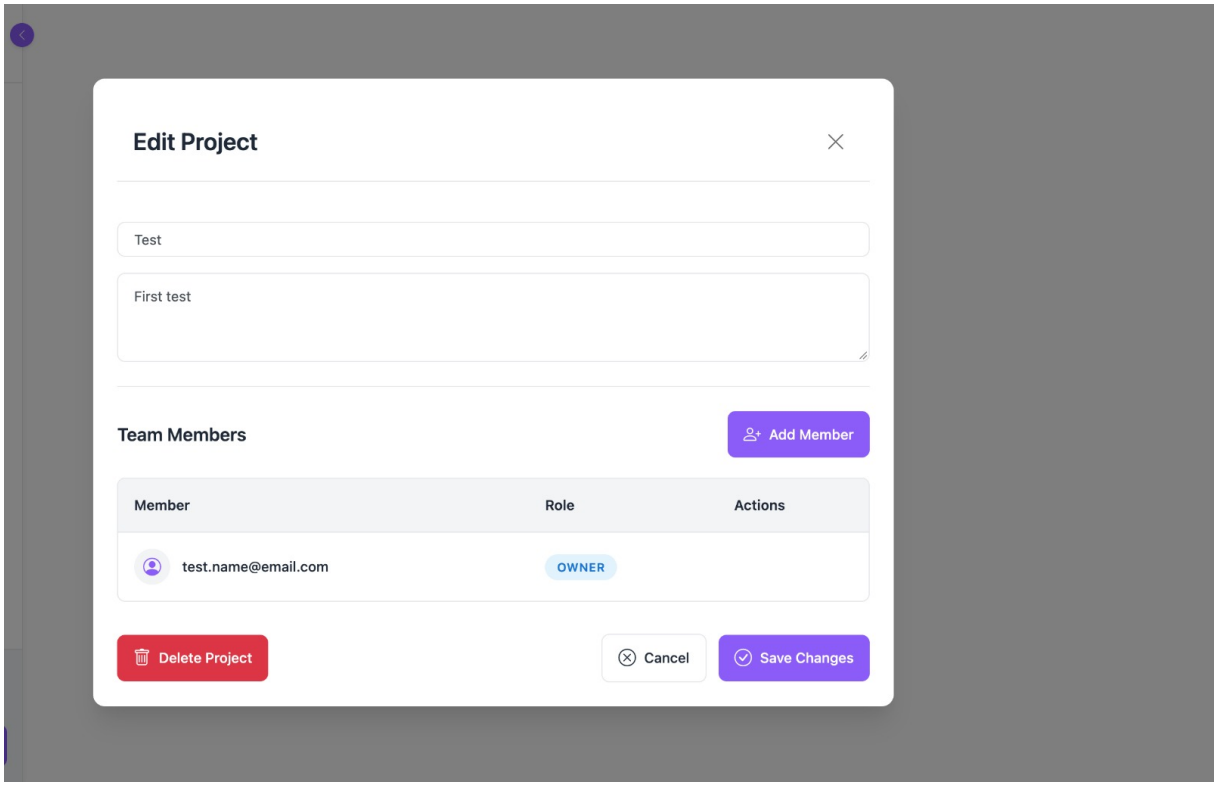


Figura 22 – *Modal* de edição de projeto e gerenciamento de time

Fonte: Elaborado pelo autor

A adição de um novo membro é realizada por meio de um *modal* secundário, apresentado na Figura 23. O *Owner* informa o e-mail do colaborador e seleciona o papel de acesso a ser atribuído (*Viewer* ou *Editor*).

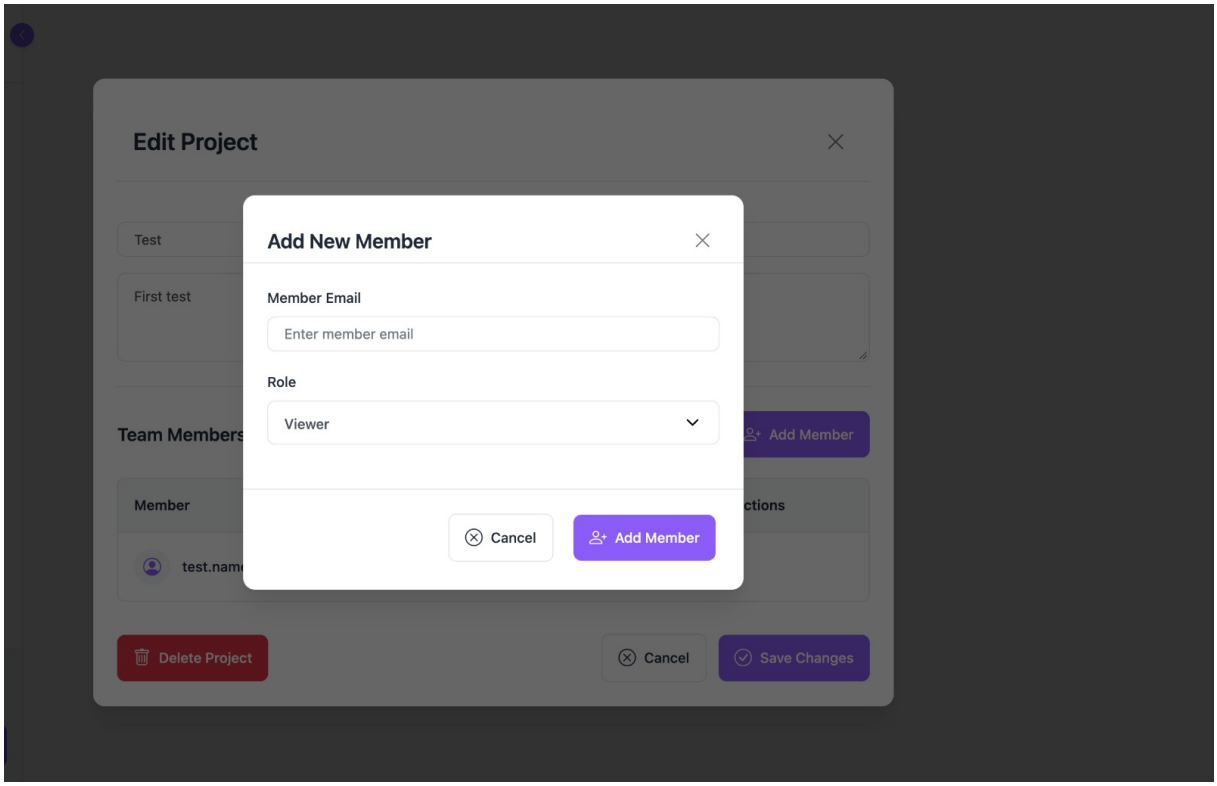


Figura 23 – *Modal* de adição de membro ao projeto

Fonte: Elaborado pelo autor

4.7.3 Editor de Diagramas

Ao selecionar um projeto no menu lateral, o usuário acessa o editor de diagramas. A Figura 24 apresenta o editor com um diagrama composto por três entidades e seus respectivos relacionamentos. A barra de ferramentas lateral disponibiliza as ações de modelagem: adição de entidade, definição do tipo de relacionamento e operações de importação e exportação SQL. No topo da tela, o papel do usuário no projeto é exibido em destaque.

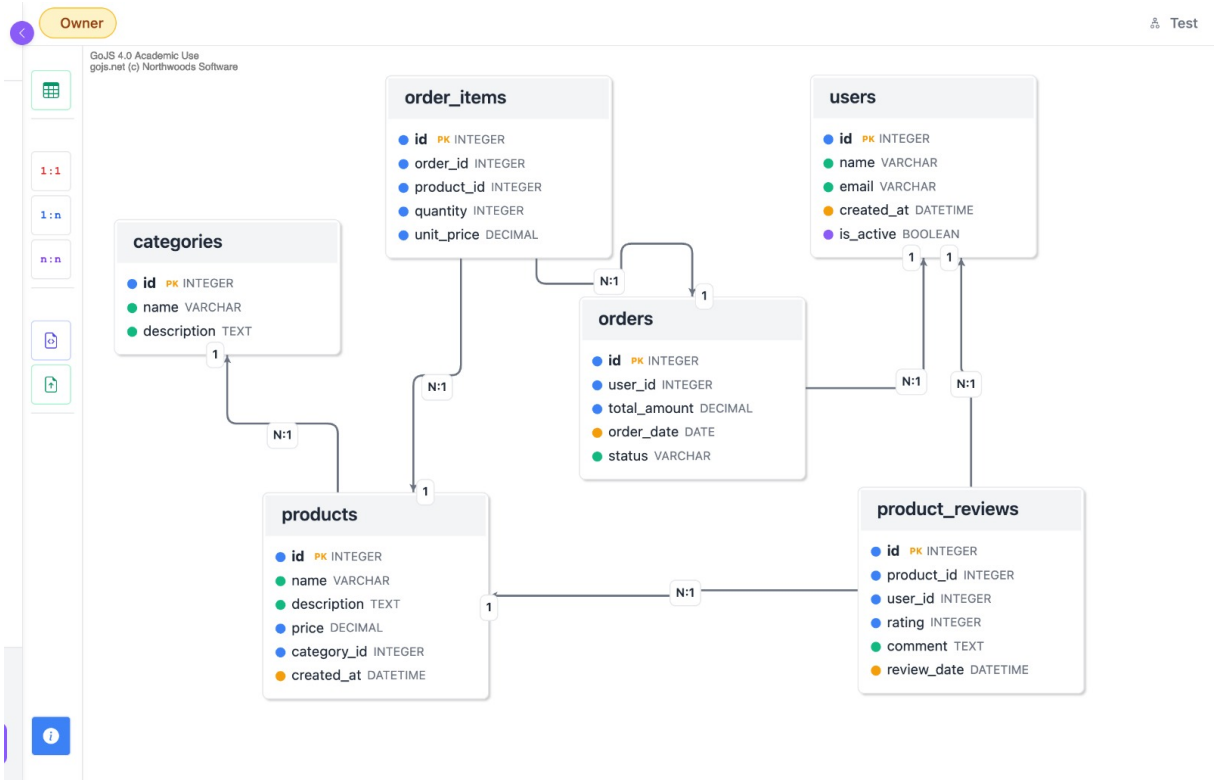


Figura 24 – Editor de diagramas com entidades e relacionamentos

Fonte: Elaborado pelo autor

Ao adicionar ou selecionar uma entidade no *canvas*, o *modal* de edição apresentado na Figura 25 é exibido. O usuário define o nome da entidade e gerencia seus atributos por meio de uma tabela interativa. Para cada atributo, é possível configurar o nome, o tipo de dado e as restrições aplicáveis: chave primária, unicidade, obrigatoriedade, auto-incremento e valor padrão. Novos atributos são adicionados por meio do botão disponível no rodapé do *modal*.

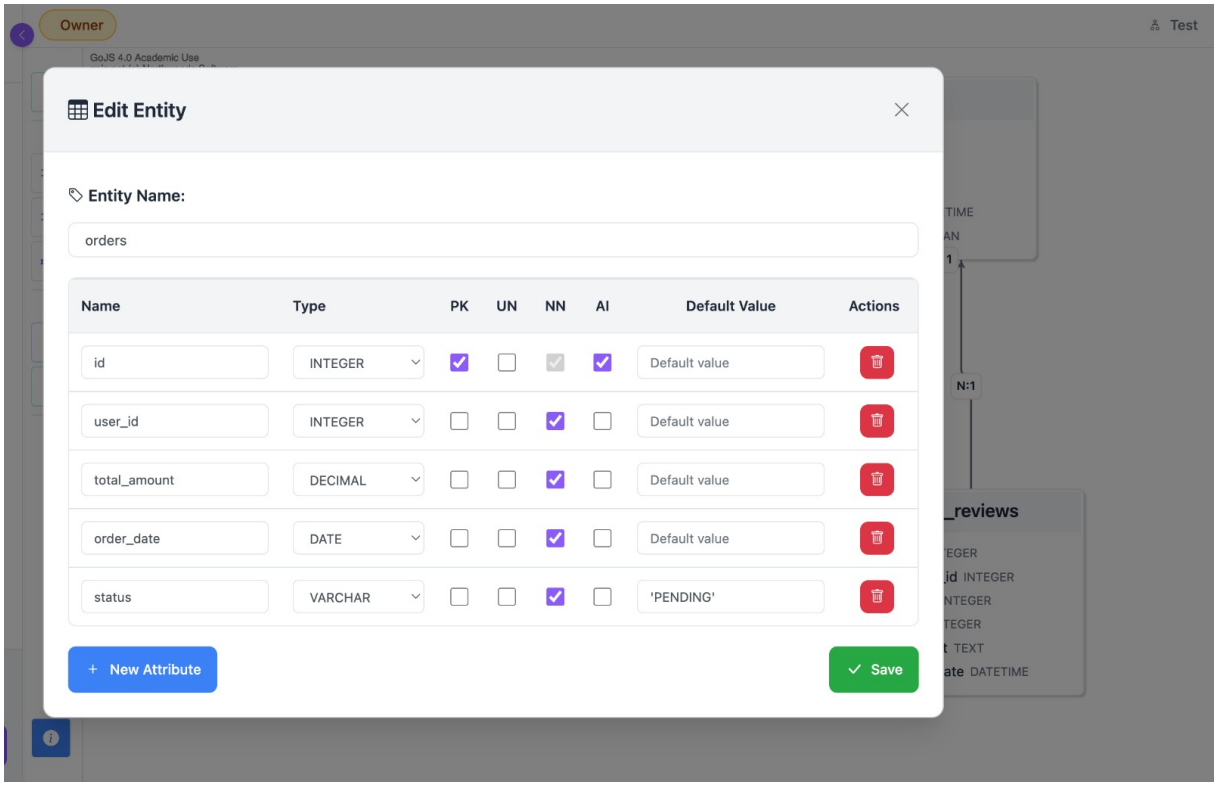


Figura 25 – *Modal* de edição de entidade

Fonte: Elaborado pelo autor

A Figura 26 exibe o mesmo diagrama com a legenda visível no *canvas*. A legenda descreve o código de cores utilizado para representar os tipos de dados dos atributos e indica os marcadores de chave primária. O diagrama reflete em tempo real as alterações realizadas por qualquer colaborador conectado ao projeto.

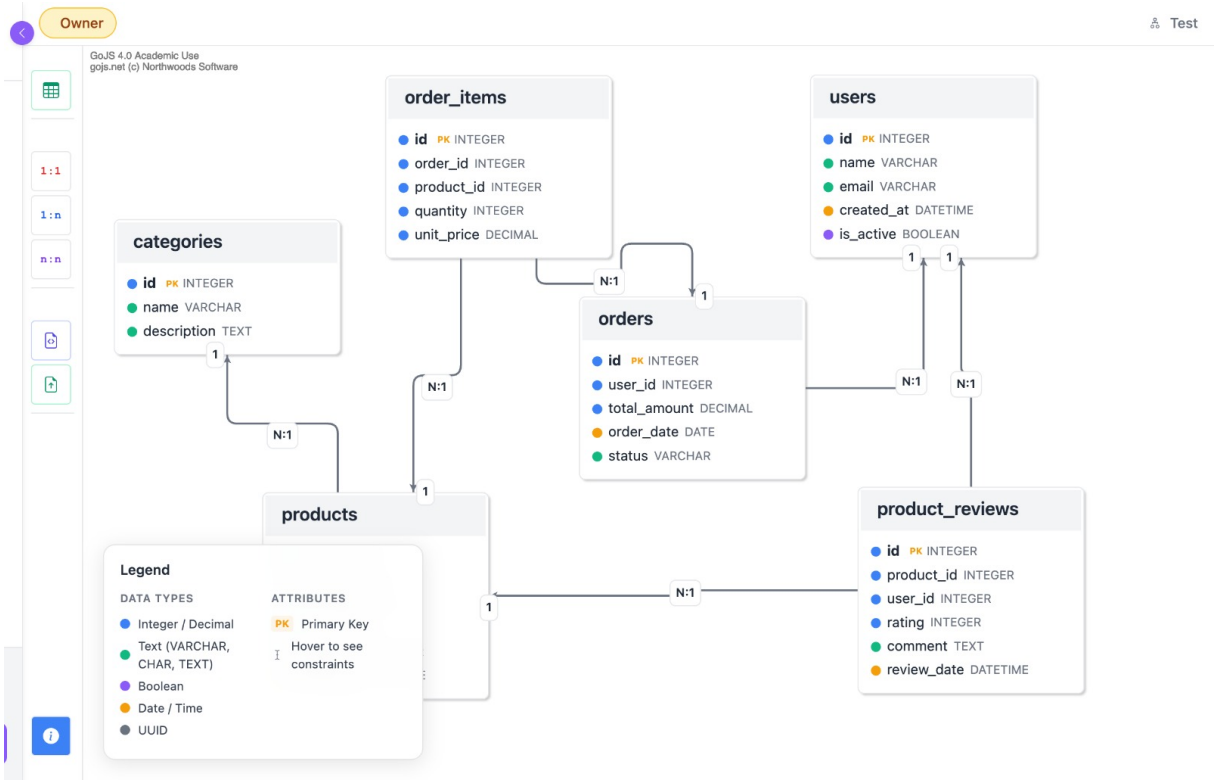


Figura 26 – Editor de diagramas com legenda de tipos de dados

Fonte: Elaborado pelo autor

Quando um usuário acessa um projeto com o papel de *Viewer*, o editor é exibido em modo somente leitura. Conforme ilustra a Figura 27, a barra lateral de ferramentas é substituída por um indicador de visualização, impedindo qualquer operação de criação ou edição no diagrama. O papel atribuído ao usuário é exibido no topo da tela.

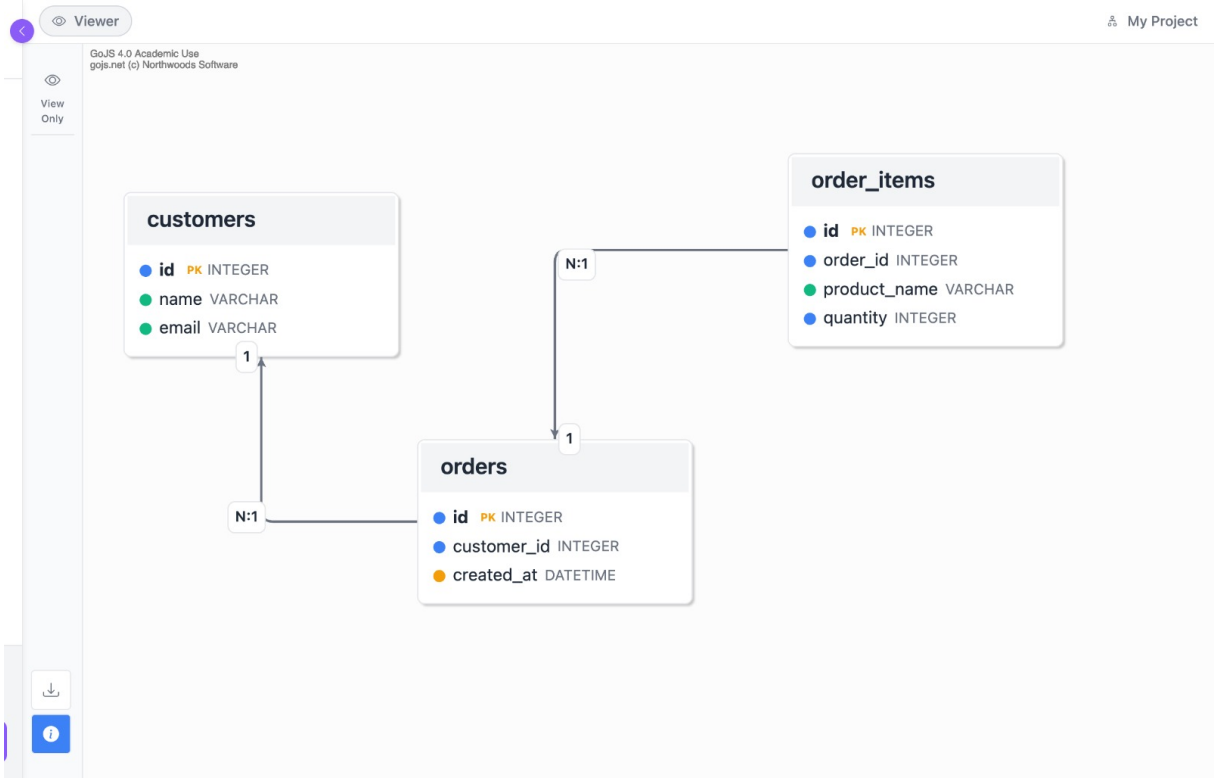


Figura 27 – Editor de diagramas no perfil *Viewer*

Fonte: Elaborado pelo autor

O mecanismo de bloqueio de entidades garante a consistência do diagrama durante sessões colaborativas simultâneas. Quando um colaborador inicia a edição de uma entidade, ela é bloqueada para os demais usuários conectados ao projeto. A Figura 28 ilustra esse comportamento: a entidade *employee* exibe uma borda tracejada indicando que está sendo editada por outro usuário, e uma notificação é exibida no topo da tela identificando o colaborador responsável pela edição.

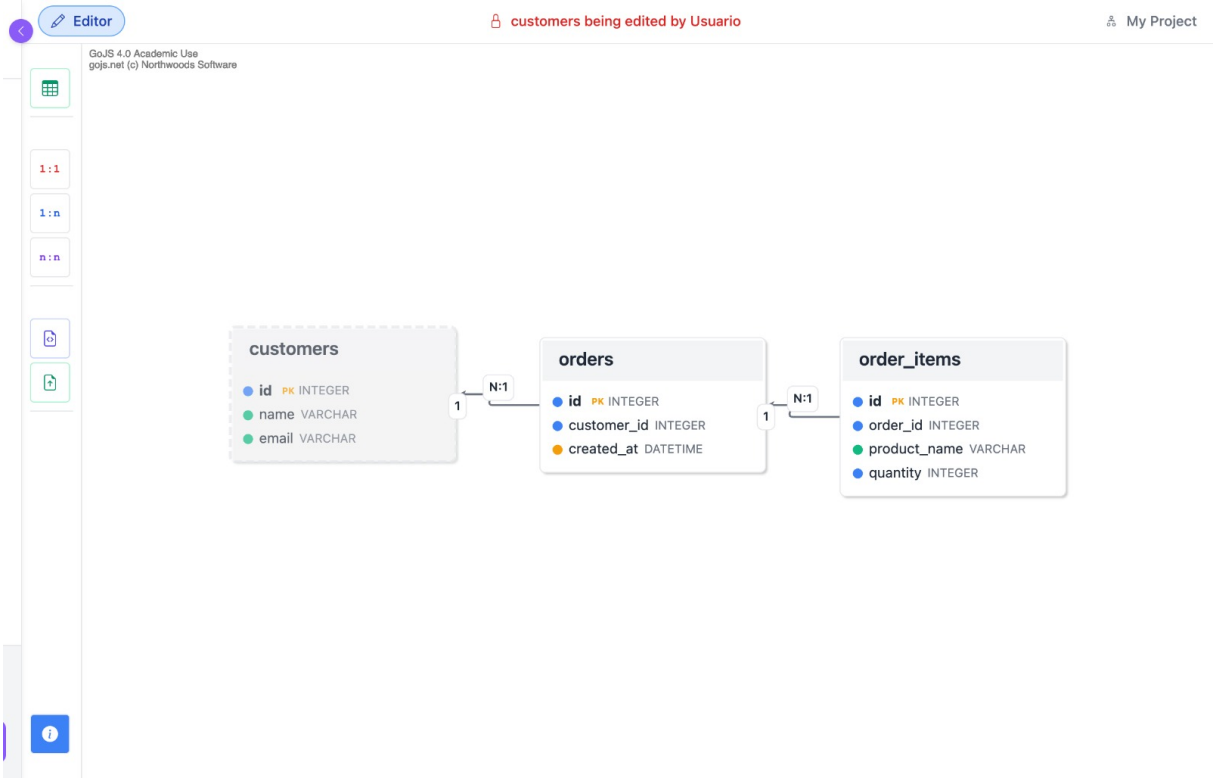


Figura 28 – Entidade bloqueada por outro colaborador durante edição simultânea

Fonte: Elaborado pelo autor

4.7.4 Importação e Exportação SQL

A importação é acionada pelo botão correspondente na barra de ferramentas lateral do editor. Conforme ilustra a Figura 29, o navegador abre a janela de seleção de arquivos do sistema operacional, na qual o usuário escolhe o *script* SQL contendo as instruções DDL a serem importadas.

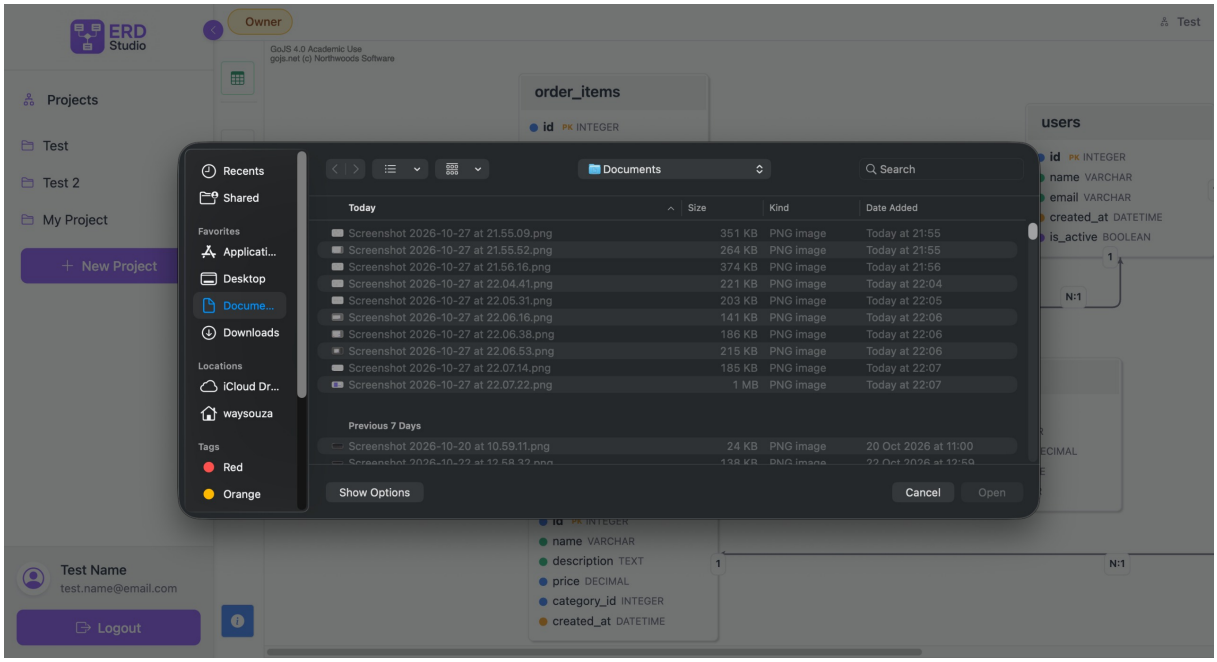


Figura 29 – Seleção do *script* SQL para importação

Fonte: Elaborado pelo autor

Após o processamento do arquivo, a aplicação informa que a importação foi concluída com sucesso e recarrega o diagrama, como mostra a Figura 30.



Figura 30 – Confirmação de importação bem-sucedida

Fonte: Elaborado pelo autor

A Figura 31 apresenta o diagrama resultante da importação de um *script* com três tabelas relacionadas: *customers*, *orders* e *order_items*. As entidades, os atributos com seus tipos de dados e os relacionamentos definidos pelas chaves estrangeiras são reconstruídos automaticamente no *canvas*, substituindo o conteúdo anterior do diagrama.



Figura 31 – Diagrama reconstruído a partir do *script* SQL importado

Fonte: Elaborado pelo autor

A exportação segue o caminho inverso. Ao acionar o botão de exportação, o diagrama é convertido em um *script* SQL e o navegador solicita o local onde o arquivo será salvo, conforme a Figura 32.

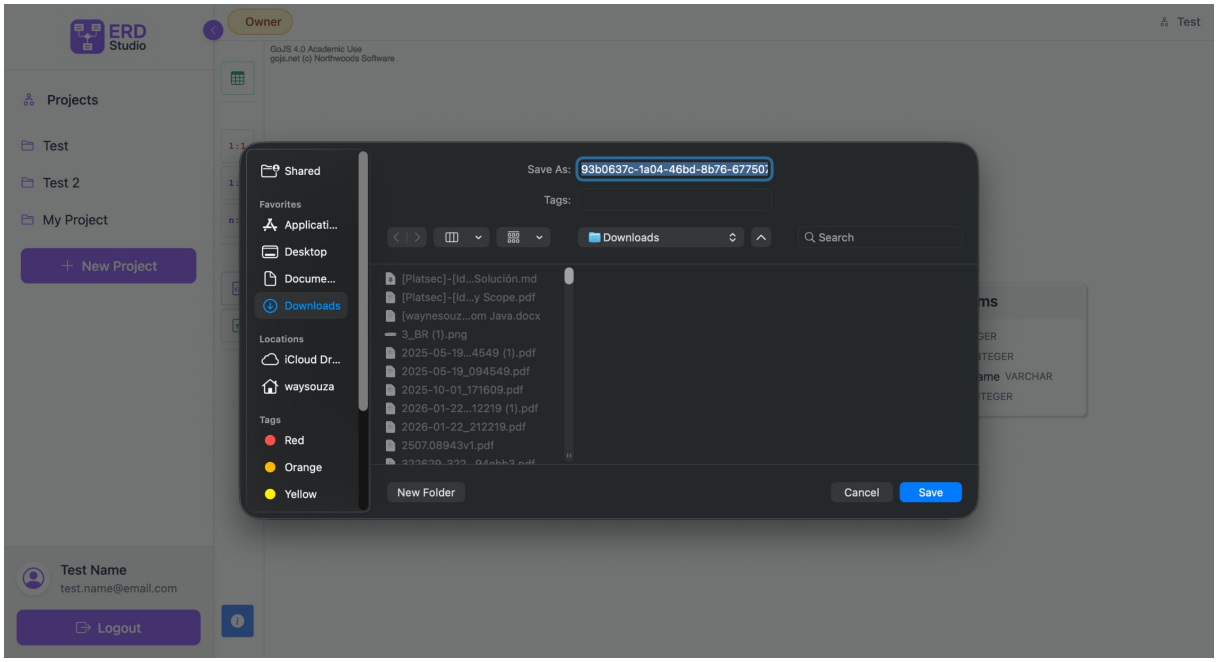


Figura 32 – Salvamento do *script* SQL exportado

Fonte: Elaborado pelo autor

O Quadro 4 apresenta o *script* gerado pela exportação do diagrama da Figura 31. As tabelas são criadas com suas chaves primárias e, em seguida, os relacionamentos são declarados como restrições de chave estrangeira por meio de instruções `ALTER TABLE`.

```
CREATE TABLE customers (
    id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    PRIMARY KEY (id)
);

CREATE TABLE orders (
    id INT AUTO_INCREMENT,
    customer_id INT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id)
);

CREATE TABLE order_items (
    id INT AUTO_INCREMENT,
    order_id INT NOT NULL,
    product_name VARCHAR(255) NOT NULL,
    quantity INT NOT NULL,
    PRIMARY KEY (id)
);

ALTER TABLE orders ADD CONSTRAINT fk_orders_customers FOREIGN KEY (
   customer_id) REFERENCES customers(id);

ALTER TABLE order_items ADD CONSTRAINT fk_order_items_orders FOREIGN KEY (
   order_id) REFERENCES orders(id);
```

Quadro 4 – *Script* SQL gerado pela exportação do diagrama

Fonte: Elaborado pelo autor

4.8 Disponibilidade do Código-Fonte e da Aplicação

O código-fonte do protótipo está organizado em dois repositórios públicos no GitHub, separados por módulo:

- ***Back-end*:** <https://github.com/waynesouza/erd-core>
- ***Front-end*:** <https://github.com/waynesouza/erd-client>

Após a implantação, a aplicação estará acessível temporariamente pelo navegador no seguinte endereço: <https://erd-client.onrender.com>.

5 Resultados e Discussões

Os resultados apresentados no capítulo anterior demonstram que o protótipo de modelador de diagramas ER colaborativo entregou um sistema funcional que integra modelagem de dados, colaboração em tempo real e controle de acesso em uma única plataforma *web*. Este capítulo discute esses resultados sob três perspectivas: a comparação com ferramentas estabelecidas no mercado, a avaliação dos testes e do desempenho do protótipo e a análise das dificuldades encontradas e das limitações do protótipo.

5.1 Comparação com Ferramentas Existentes

A fim de contextualizar o protótipo desenvolvido, a Tabela 12 apresenta uma comparação entre o protótipo e as três ferramentas de modelagem ER analisadas na fundamentação teórica: brModeloWeb, SqlDBM e Vertabelo.

Tabela 12 – Comparação entre o protótipo e ferramentas de modelagem ER

Funcionalidade	**Protótipo**	**brModeloWeb**	**SqlDBM**	**Vertabelo**
Modelagem ER	Sim	Sim	Sim	Sim
Colaboração em tempo real	Sim	Não	Sim	Sim
Controle de versão	Não	Não	Sim	Sim
Exportação SQL	Sim	Sim	Sim	Sim
Importação SQL	Sim	Não	Sim	Sim
Baseado em *web*	Sim	Sim	Sim	Sim
Gratuito	Sim	Sim	Não	Não

Fonte: Elaborado pelo autor

A análise comparativa evidencia que, dentre as ferramentas analisadas, o protótipo desenvolvido distingue-se por reunir, em uma plataforma *web* e de forma gratuita para o usuário final, a colaboração em tempo real com suporte completo a importação e exportação SQL. O brModeloWeb, embora gratuito e baseado em *web*, limita a colaboração ao compartilhamento dos modelos, sem suporte à edição simultânea entre múltiplos usuários. O SqlDBM e o Vertabelo oferecem colaboração em tempo real e operações DDL mais abrangentes, porém não dispõem de plano gratuito, sendo acessíveis apenas mediante assinatura paga. Ambas oferecem ainda controle de versão dos modelos, recurso que o protótipo não contempla e que é tratado como uma de suas limitações na Seção 5.6. O

protótipo ocupa essa intersecção ao combinar colaboração em tempo real, operações SQL e acesso sem custo para o usuário final, ressalvada a licença da biblioteca de diagramação discutida na Seção 5.6, alinhando-se ao diagnóstico identificado na justificativa do trabalho: a carência de uma ferramenta *web*, acessível e colaborativa para a modelagem de diagramas ER.

5.2 Avaliação dos Testes

A cobertura de código de 100% indica que todo o código foi executado pelos testes unitários, mas não o quanto do seu comportamento eles verificam (INOZEMTSEVA; HOLMES, 2014). Os resultados deste trabalho mostram essa diferença de duas formas. A primeira é o que cada nível de teste é capaz de verificar. Nos testes unitários, as partes de que cada unidade depende, como a distribuição das mensagens aos colaboradores e a consulta às permissões no banco de dados, são substituídas por versões simuladas. Por isso, esses testes não verificam se uma alteração de fato chega aos demais membros do projeto, nem se as permissões são consultadas corretamente quando a requisição chega pelo canal em tempo real. Esses comportamentos, que sustentam o objetivo central do trabalho, são verificados pelos testes de integração e de sistema, que executam a aplicação completa. Da mesma forma, situações como o acesso ao canal fora do fluxo normal da interface e dois pedidos simultâneos de bloqueio da mesma tabela só são verificadas por testes escritos especificamente para elas: os testes de integração e um teste unitário em que várias execuções simultâneas disputam o mesmo bloqueio.

A segunda forma é o teste de mutação, que quantifica a diferença entre executar e verificar. No *back-end*, 93,4% dos mutantes foram detectados. Os sobreviventes apontam trechos executados pelos testes cujo resultado não é conferido, como o preenchimento de campos na importação de *scripts* SQL. Entre eles, merece atenção o mutante que faz a verificação de que um usuário é o proprietário de um projeto, usada na edição do projeto e na gestão dos seus membros, retornar sempre verdadeiro: uma regressão nessa regra não seria detectada pelos testes unitários. No protótipo, ela é compensada por uma segunda verificação de propriedade, feita antes de a requisição chegar a essa regra. O outro mutante de verificação de permissão está em um método que não é usado pela aplicação.

No *front-end*, o escore foi menor, de 80,7%. Mais da metade dos mutantes de texto sobreviventes altera mensagens escritas no console do navegador, que não fazem parte do comportamento da aplicação e que, portanto, nenhum teste deveria detectar. Os demais concentram-se no componente do editor de diagramas, em trechos que os testes executam sem conferir integralmente o resultado, como as cores associadas aos tipos de dados e os nomes usados para localizar a chave estrangeira a ser removida quando um relacionamento é excluído.

Os três níveis de teste se mostraram, portanto, complementares. A cobertura de 100% garantiu que todo o código fosse exercitado ao menos uma vez, os testes de integração e de sistema verificaram o comportamento das partes em conjunto e da forma como os usuários o percebem, e o teste de mutação indicou onde os testes unitários devem ser reforçados.

5.3 Avaliação do Desempenho

No ambiente local, a latência de propagação foi de cerca de 8 milissegundos, com o percentil 95 abaixo de 10 milissegundos para grupos de 2 a 10 usuários (Tabela 10). Como esse ambiente não envolve tráfego pela rede externa, esse valor representa essencialmente o custo de processamento de uma alteração no servidor, e está abaixo do limite de 0,1 segundo a partir do qual, segundo NIELSEN (1993), uma resposta deixa de ser percebida como imediata. No ambiente publicado, as medianas ficaram entre 169 e 185 milissegundos, da mesma ordem do tempo de ida e volta de uma requisição simples ao servidor (mediana de 204 milissegundos). A diferença entre os dois ambientes corresponde, portanto, à distância de rede entre o Brasil e o servidor, nos Estados Unidos, e não ao processamento. O aumento de 2 para 10 usuários não aumentou a mediana, mas ampliou a dispersão, com picos isolados de até 482 milissegundos.

A latência percebida pelo usuário, por outro lado, ficou acima de 1 segundo nos dois ambientes: 1,53 segundo no local e 1,71 segundo no publicado (Tabela 11). Quase todo esse tempo corresponde ao intervalo de 1,5 segundo que antecede o envio automático das alterações; descontado esse intervalo, restam cerca de 32 milissegundos no ambiente local e 207 milissegundos no publicado. O fator que determina o que o usuário percebe é, assim, uma escolha de projeto, e não a comunicação em tempo real: o intervalo agrupa as alterações feitas em sequência, mas faz com que cada alteração leve mais de 1 segundo para chegar aos demais colaboradores, acima do limite em que, segundo NIELSEN (1993), o atraso deixa de interromper o fluxo de raciocínio do usuário.

A liberação das tabelas bloqueadas por um usuário que se desconecta (caso CT-07) apresentou a maior diferença entre os ambientes: de 0,07 a 0,17 segundo no local e de 4,5 a 31,1 segundos no publicado. A tabela só é liberada quando o servidor percebe que a conexão terminou. No ambiente publicado, a conexão passa pelos servidores intermediários da plataforma, e os tempos observados indicam que o servidor demora a ser informado do seu encerramento. A causa exata não foi determinada neste trabalho, pois exigiria os registros internos da plataforma. Nesse intervalo, os demais colaboradores veem a tabela como bloqueada, mas a liberação automática após 5 minutos limita o pior caso.

5.4 Ameaças à Validade

Os resultados dos testes e das medições estão sujeitos a algumas limitações. Os testes de sistema foram executados automaticamente apenas no Google Chrome; no Mozilla Firefox, a interface foi verificada manualmente. O roteiro cobre os casos de uso que envolvem a colaboração; os demais, como a gestão de projetos e de membros e a exportação de *scripts* SQL, foram verificados apenas pelos testes unitários e pelos testes manuais. As medições de latência usaram no máximo 10 usuários simultâneos, todos simulados a partir de um único computador, e um diagrama com uma única tabela. Como o servidor distribui o diagrama completo a cada alteração, diagramas maiores produzem mensagens maiores e podem apresentar latências maiores. No ambiente publicado, as medições foram feitas em um único dia, no plano gratuito e em uma única região da plataforma, e podem variar com o horário e com a carga da plataforma.

5.5 Dificuldades e Lições Aprendidas

O desenvolvimento do protótipo apresentou desafios técnicos relevantes. A sincronização do estado do diagrama em um cenário de múltiplos colaboradores simultâneos constituiu a maior complexidade do projeto. Garantir que cada alteração chegasse a todos os colaboradores e que alterações simultâneas não se perdessem exigiu iterações de projeto sobre a estratégia de *locks* em memória e sobre o formato das mensagens trocadas. Enviar o diagrama inteiro a cada gravação, a forma mais simples de sincronizá-lo, faria com que a última gravação apagasse as alterações feitas por outros colaboradores em outras tabelas no mesmo intervalo. Por isso, adotou-se o envio apenas dos elementos alterados, combinados pelo servidor, comportamento verificado pelo caso CT-05. As travas com liberação automática atenderam aos cenários verificados nos testes de sistema (casos CT-03 e CT-07).

A integração da biblioteca GoJS com o mecanismo de detecção de mudanças do Angular também demandou atenção. Alterações no modelo do diagrama originadas externamente, via WebSocket, precisaram ser aplicadas ao *canvas* da GoJS de maneira a não disparar ciclos redundantes de atualização, o que exigiu controle explícito das zonas de execução do Angular.

Por fim, a adoção da persistência poliglota (PostgreSQL e MongoDB) permitiu armazenar cada tipo de dado no modelo mais próximo da sua estrutura: os usuários, os projetos e as permissões, com integridade referencial, no banco relacional, e cada diagrama como um único documento no banco não relacional. No entanto, essa decisão impôs a necessidade de gerenciar dois sistemas de banco de dados distintos, tanto no ambiente de desenvolvimento local quanto na configuração do Docker Compose para implantação, aumentando a complexidade operacional do projeto.

5.6 Limitações do Protótipo

Apesar dos resultados alcançados, o protótipo apresenta limitações que definem seu escopo, tanto em relação a uma solução de produção quanto ao alcance da análise realizada.

A primeira limitação refere-se à ausência de um histórico de versões do diagrama. Na versão atual, o estado do diagrama é substituído a cada conjunto de atualizações persistidas, sem preservação de versões anteriores. Isso impede que os colaboradores desfaçam alterações em nível colaborativo ou inspecionem o histórico de mudanças do projeto, funcionalidade presente em ferramentas maduras de colaboração. A implementação de um mecanismo de versionamento, como o armazenamento de *snapshots* incrementais, constitui uma evolução natural do protótipo.

A segunda limitação diz respeito à licença comercial da biblioteca GoJS. A GoJS exige a aquisição de uma licença para uso em ambiente de produção, disponibilizando apenas uma licença de avaliação para fins acadêmicos e de prototipagem. A adoção do protótipo em contexto comercial estaria sujeita a esses custos de licenciamento, ponto já identificado na investigação apresentada na Seção 4.3. Em trabalhos futuros, a substituição desta biblioteca por uma alternativa de código aberto com capacidades equivalentes seria um caminho viável para eliminar essa restrição.

A terceira limitação refere-se ao suporte restrito ao dialeto MySQL nas operações DDL e na construção dos diagramas. Tanto a exportação quanto a importação de *scripts* SQL foram implementadas para o dialeto MySQL (Oracle, 2024b), não contemplando outros SGBDs amplamente utilizados. Cada um desses sistemas possui particularidades sintáticas próprias em suas instruções DDL que não são necessariamente compatíveis entre si. Dessa forma, *scripts* gerados pelo protótipo podem requerer adaptações para funcionar corretamente em ambientes que não utilizem o MySQL, e *scripts* importados provenientes de outros SGBDs podem não ser interpretados corretamente pelo protótipo.

A quarta limitação refere-se ao alcance da análise comparativa, restrita a três ferramentas (brModeloWeb, SqlDBM e Vertabelo), selecionadas por serem aplicações *web* com suporte a diagramas ER e consultadas no período de realização deste trabalho. Não se pretende, portanto, afirmar a inexistência de outras soluções com capacidades equivalentes. Além disso, as funcionalidades e os planos de assinatura dessas ferramentas podem mudar ao longo do tempo, e a ampliação do conjunto de ferramentas comparadas constitui um possível desdobramento deste trabalho.

A quinta limitação refere-se à granularidade da combinação de alterações. O servidor combina as alterações elemento a elemento, de modo que alterações simultâneas em um mesmo elemento seguem a regra de que a última gravação prevalece. Na edição de tabelas, a trava impede essa situação, mas o reposicionamento de elementos não adquire trava: se dois

usuários arrastarem a mesma tabela ao mesmo tempo, prevalece a última posição gravada. Além disso, o servidor distribui o diagrama completo a cada alteração, e o tamanho das mensagens cresce com o tamanho do diagrama.

A sexta limitação refere-se aos tempos de resposta observados na Seção 5.3. O intervalo de 1,5 segundo que antecede o envio automático das alterações faz com que a latência percebida pelo usuário supere 1 segundo, e, no ambiente publicado, a liberação das tabelas bloqueadas por um usuário que se desconecta levou até 31,1 segundos.

5.7 Trabalhos Futuros

A análise da arquitetura do protótipo e os resultados dos testes apontam cinco direções prioritárias para a evolução do sistema rumo a um ambiente de produção.

5.7.1 Camada de Cache Distribuído

O mecanismo de *locks* de colaboração implementado no protótipo armazena o estado de edição em uma estrutura de dados mantida na memória local do servidor, assim como o controle que aplica as alterações de cada projeto uma de cada vez. Essa abordagem só é correta enquanto a aplicação opera em uma única instância, e se torna um ponto de falha ao escalar horizontalmente: servidores distintos não compartilham o mesmo estado de *locks*, o que pode originar conflitos de edição entre colaboradores atendidos por máquinas diferentes. A adoção do Redis (um repositório de dados em memória) como armazenamento compartilhado entre instâncias solucionaria esse problema, ao tornar cada *lock* visível a todo o conjunto de servidores. Além do gerenciamento de *locks*, o Redis possibilitaria o *cache* de consultas frequentes, como listas de projetos e perfis de usuários, reduzindo a carga sobre o banco de dados relacional e melhorando o tempo de resposta da aplicação.

5.7.2 Escalabilidade Horizontal do Canal WebSocket

O *broker* de mensagens WebSocket utilizado no protótipo opera como um componente interno a cada instância do servidor. Em consequência, mensagens originadas em uma instância são entregues exclusivamente aos clientes conectados àquele mesmo servidor: em um ambiente com múltiplos servidores, colaboradores em instâncias distintas deixariam de receber as atualizações do diagrama uns dos outros, comprometendo a experiência colaborativa. A solução consiste em substituir o *broker* embutido por um serviço de mensageria externo (como o RabbitMQ), que centralize o roteamento das mensagens entre todas as instâncias e garanta que qualquer atualização seja propagada a todos os colaboradores independentemente do servidor ao qual cada um está conectado. Essa melhoria, combinada com a containerização da aplicação, consolidaria a infraestrutura necessária para suportar carga de produção real com múltiplas réplicas.

5.7.3 Tratamento de Erros e Resiliência

O protótipo trata exceções de forma pontual e sem padronização, o que faz com que falhas inesperadas retornem respostas inconsistentes ao cliente. A implementação de um tratador global de exceções, complementado por uma hierarquia de exceções de domínio específicas, tornaria o comportamento da API previsível e facilitaria o diagnóstico de problemas. Em paralelo, a adoção do padrão *circuit breaker* aumentaria a resiliência do sistema diante de indisponibilidades temporárias: em vez de propagar falhas em cascata quando um serviço externo (como o banco de dados de documentos) está inacessível, o *circuit breaker* interrompe as chamadas ao componente falho e aciona um comportamento alternativo, permitindo que o restante da aplicação continue operando de forma degradada, mas estável, até que a dependência seja restabelecida.

5.7.4 Tempos de Resposta da Colaboração

Dois ajustes reduziriam os tempos discutidos na Seção 5.3. O primeiro é diminuir o intervalo que antecede o envio automático das alterações: como as alterações já são combinadas elemento a elemento pelo servidor, envios mais frequentes não aumentam o risco de sobrescrita. O segundo é fazer com que os clientes enviem sinais periódicos de atividade e que o servidor encerre as conexões que deixarem de enviá-los, de modo que o servidor perceba por conta própria uma conexão encerrada, sem depender dos servidores intermediários da plataforma de hospedagem.

5.7.5 Reforço dos Testes

Os mutantes sobreviventes do teste de mutação indicam os trechos em que os testes unitários devem passar a verificar os resultados, e não apenas executá-los, a começar pelas verificações de permissão. O roteiro de testes de sistema pode ser estendido aos demais casos de uso, como a gestão de projetos e de membros e a exportação de *scripts* SQL, e executado também no Mozilla Firefox.

6 Considerações Finais

O presente trabalho cumpriu com seus objetivos, confirmando a hipótese formulada na metodologia. A plataforma resultante integra, em uma única solução gratuita, a modelagem interativa de esquemas de banco de dados, a colaboração em tempo real entre múltiplos usuários e operações de importação e exportação SQL, conjunto de capacidades que, dentre as ferramentas analisadas (Tabela 12), não encontra equivalente gratuito.

Os cinco objetivos específicos definidos na introdução foram atendidos. A Tabela 13 relaciona cada um deles aos casos de uso que o detalham, descritos no Anexo A, e às figuras, tabelas e quadros que evidenciam o seu atendimento. O segundo objetivo, por ser de natureza investigativa, não corresponde a um caso de uso.

Tabela 13 – Relação entre os objetivos específicos, os casos de uso e as evidências

Objetivo específico	**Casos de uso**	**Evidências**
1. Modelar e persistir os elementos do diagrama	CSU-06, CSU-07 e CSU-13 a CSU-22	Figuras 16 e 24 a 26; Quadros 1 e 2; Tabela 9 (CT-06)
2. Investigar a biblioteca GoJS	–	Tabela 4
3. Permitir a edição simultânea do mesmo diagrama por múltiplos usuários	CSU-07, CSU-13 e CSU-14	Figuras 17 e 28; Tabelas 9 (CT-01, CT-02, CT-03, CT-05 e CT-07), 10 e 11
4. Garantir que apenas usuários autorizados acessem e modifiquem os diagramas	CSU-01 a CSU-05 e CSU-08 a CSU-12	Figuras 18, 19 a 23 e 27; Tabela 9 (CT-04)
5. Importar e exportar diagramas por meio de *scripts* SQL	CSU-23 e CSU-24	Figuras 29 a 32; Quadro 4

Fonte: Elaborado pelo autor

O primeiro objetivo, de modelar e persistir os elementos do diagrama, foi atendido pelo editor interativo, que registra tabelas, atributos, tipos de dados, relacionamentos e posicionamentos e preserva o diagrama entre as sessões de trabalho. O segundo, de investigar a biblioteca GoJS, foi cumprido pela análise apresentada na Seção 4.3, que fundamentou a sua adoção para a exibição e a edição interativa dos diagramas. O terceiro, de permitir que múltiplos usuários trabalhem simultaneamente no mesmo diagrama, foi

atendido pela comunicação em tempo real entre os colaboradores, que propaga as alterações de cada um aos demais e as combina sem que uma sobrescreva a outra. O quarto, de garantir que apenas usuários autorizados acessem e modifiquem os diagramas, foi atendido pela autenticação por *token* e pela verificação do perfil de cada usuário no projeto, tanto nas requisições ao servidor quanto na comunicação em tempo real. O quinto, de importar e exportar diagramas por meio de *scripts* SQL, foi atendido pela construção do diagrama a partir de um *script* no dialeto MySQL e pela geração do *script* correspondente a um diagrama modelado.

As limitações identificadas, em especial nos tempos de resposta da colaboração, foram discutidas na Seção 5.6 e orientam os trabalhos futuros propostos.

O ganho proporcionado por este trabalho se manifesta em duas dimensões. Do ponto de vista técnico, o protótipo serve como referência prática para a construção de ferramentas colaborativas baseadas em comunicação em tempo real com editores visuais interativos em ambiente *web*. Do ponto de vista da área, o trabalho contribui para reduzir a lacuna identificada na justificativa: a carência de uma ferramenta colaborativa, baseada em *web* e sem barreiras de custo para a modelagem de diagramas ER, atendendo às demandas de equipes de desenvolvimento distribuídas.

Referências

ALVES, L. N. et al. Inglês: o requisito fundamental na área da tecnologia da informação e comunicação. *Faculdade de Tecnologia de Botucatu*, 2022. Citado na página 33.

AQUILES, A.; FERREIRA, R. *Controlando Versões com Git e GitHub*. [S.l.]: Casa do Código, 2014. 199 p. Citado na página 26.

BOEHM, B. W. A spiral model of software development and enhancement. *Computer*, v. 21, n. 5, p. 61–72, 1988. Citado na página 22.

brModeloWeb. *brModeloWeb: ferramenta online de modelagem de banco de dados*. 2026. Disponível em: <https://www.brmodeloweb.com>. Citado na página 17.

BULGAKOVA, O. S.; ZOSIMOV, V. V.; POPRAVKIN, P. D. Storing jwt token in local variables. *Control Systems & Computers*, 2020. Citado na página 27.

Client IO. *JointJS Demos*. 2026. Disponível em: <https://www.jointjs.com/demos>. Citado na página 46.

Client IO. *JointJS: JavaScript diagramming library*. 2026. Disponível em: <https://www.jointjs.com/>. Citado na página 46.

DEMILLO, R. A.; LIPTON, R. J.; SAYWARD, F. G. Hints on test data selection: Help for the practicing programmer. *Computer*, v. 11, n. 4, p. 34–41, 1978. Citado na página 31.

Docker. *About Docker*. 2024. Disponível em: <https://www.docker.com/company/>. Citado na página 20.

ELMASRI, R.; NAVATHE, S. *Fundamentals of Database Systems, 7th edition*. Bengaluru: Pearson India, 2017. 1272 p. Citado 2 vezes nas páginas 13 e 16.

FILIPPO, D. *Controle de Concorrência em Editores Colaborativos*. 2025. Disponível em: <https://www-di.inf.puc-rio.br/~endler/courses/DA/Monografias/05/EdicaoColaborativa-Denise.pdf>. Acesso em: 09 ago. 2025. Citado 2 vezes nas páginas 16 e 28.

FUKS, H.; RAPOSO, A.; GEROSA, M. Engenharia de groupware: Desenvolvimento de aplicações colaborativas. *XXI Jornada de Atualização em Informática, Anais do XXII Congresso da Sociedade Brasileira de Computação*, 2002. Disponível em: <https://web.tecgraf.puc-rio.br/~abraposo/pubs/JAI2002/JAI2002.pdf>. Acesso em: 05 ago. 2023. Citado 2 vezes nas páginas 13 e 16.

Git. *About Branching and Merging*. 2025. Disponível em: <https://git-scm.com/about/branching-and-merging>. Citado na página 20.

GitHub. *About GitHub*. 2024. Disponível em: <https://github.com/about>. Citado na página 20.

GitHub. *About npm*. 2024. Disponível em: <https://www.npmjs.com/about>. Citado na página 20.

GitHub Copilot. *Seu agente de codificação de IA*. 2026. Disponível em: <https://github.com/features/copilot>. Citado na página 20.

Google. *What is Angular?* 2022. Disponível em: <https://v14.angular.io/guide/what-is-angular>. Citado na página 20.

Google. *Google Chrome*. 2025. Disponível em: <https://www.google.com/chrome/>. Citado na página 21.

H2 Database Engine. *H2 Database Engine*. 2024. Disponível em: <https://www.h2database.com/>. Citado na página 21.

HIPÓLITO, E. *Dealing with Concurrency: Challenges and Solutions*. 2024. Disponível em: <https://medium.com/@estiverherrerahipolito/dealing-with-concurrency-challenges-and-solutions-fe8146b45c28>. Acesso em: 09 ago. 2025. Citado na página 28.

HSU, T. et al. Self-efficacy and behavior patterns of learners using a real-time collaboration system developed for group programming. *International Journal of Computer-Supported Collaborative Learning*, 2021. Disponível em: <https://link.springer.com/article/10.1007/s11412-021-09357-3>. Acesso em: 30 out. 2023. Citado 2 vezes nas páginas 13 e 16.

IBM. *O que é MongoDB?* 2021. Disponível em: <https://www.ibm.com/br-pt/think/topics/mongodb>. Citado na página 20.

INOUE, C. R. et al. Uso de ferramentas colaborativas em trabalho coletivo. *XVIII Seminário Nacional de Bibliotecas Universitárias*, 2014. Citado 2 vezes nas páginas 13 e 16.

INOZEMTSEVA, L.; HOLMES, R. Coverage is not strongly correlated with test suite effectiveness. In: *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Hyderabad: ACM, 2014. p. 435–445. Citado 2 vezes nas páginas 29 e 71.

IVANČEVIĆ, T. K.; JEŽIĆ, T. J.; LOKNAR, N. S. A cross-device and cross-os benchmark of modern web animation systems. *Journal of Imaging*, v. 12, n. 1, p. 45, 2026. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12843483/>. Acesso em: 19 out. 2026. Citado 2 vezes nas páginas 46 e 47.

JaCoCo. *JaCoCo Java Code Coverage Library*. 2024. Disponível em: <https://www.jacoco.org/jacoco/>. Citado na página 21.

Jasmine. *Jasmine*. 2024. Disponível em: <https://jasmine.github.io/>. Citado na página 21.

JetBrains. *IntelliJ IDEA*. 2024. Disponível em: <https://www.jetbrains.com/pt-br/idea/>. Citado na página 20.

JetBrains. *WebStorm*. 2024. Disponível em: <https://www.jetbrains.com/pt-br/webstorm/>. Citado na página 20.

JUnit. *JUnit 5*. 2023. Disponível em: <https://junit.org/>. Citado na página 21.

Karma. *Karma - Spectacular Test Runner for JavaScript*. 2024. Disponível em: <https://karma-runner.github.io/>. Citado na página 21.

LOPES, P. et al. *Choosing the Right Git Workflow: A Comparative Analysis of Trunk-based vs. Branch-based Approaches*. 2025. Disponível em: <https://arxiv.org/pdf/2507.08943>. Acesso em: 06 ago. 2025. Citado na página 26.

MARTIN, R. C. *Código Limpo: Habilidades Práticas do Agile Software*. Rio de Janeiro: Alta Books, 2009. 425 p. Citado 2 vezes nas páginas 25 e 34.

Microsoft. *Playwright: Fast and reliable end-to-end testing for modern web apps*. 2024. Disponível em: <https://playwright.dev/>. Citado na página 21.

Mockito. *Mockito*. 2025. Disponível em: <https://site.mockito.org/>. Citado na página 21.

MOHAGHEGHI, P.; CONRADI, R. Quality, productivity and economic benefits of software reuse: a review of industrial studies. *Empirical Software Engineering*, v. 12, n. 5, p. 471–516, 2007. Disponível em: <https://link.springer.com/article/10.1007/s10664-007-9040-x>. Acesso em: 19 out. 2026. Citado na página 45.

Mozilla. *Firefox*. 2026. Disponível em: <https://www.mozilla.org/pt-BR/firefox/new/>. Citado na página 21.

NETO, M. B. S.; MELLO, R. S. brmodeloweb: Ten years of history and a new extension for the logical design of nosql databases. In: *Anais Estendidos do XLI Simpósio Brasileiro de Banco de Dados (SBBD)*. [S.l.]: Sociedade Brasileira de Computação, 2026. p. 215–220. Disponível em: <https://sol.sbc.org.br/index.php/sbbd_estendido/article/view/44134>. Acesso em: 21 out. 2026. Citado na página 17.

NIELSEN, J. *Usability Engineering*. Boston: Academic Press, 1993. Citado 2 vezes nas páginas 32 e 72.

Northwoods Software Corporation. *GoJS and Angular*. 2024. Disponível em: <https://gojs.net/latest/intro/angular.html>. Citado 2 vezes nas páginas 20 e 46.

Northwoods Software Corporation. *GoJS Deployment*. 2026. Disponível em: <https://gojs.net/latest/intro/deployment.html>. Citado na página 47.

Northwoods Software Corporation. *GoJS Entity Relationship Sample*. 2026. Disponível em: <https://gojs.net/latest/samples/entityRelationship.html>. Citado na página 46.

OLIVEIRA, C.; SANTANA, G.; REIS, G. *Live Saver Locator: Sistema de Chamado e Controle de Ambulâncias*. 2024. Disponível em: <http://ric-cps.eastus2.cloudapp.azure.com/bitstream/123456789/32958/1/ads_2024_1_caiorodrigodeoliveira_livesaverlocator.pdf>. Acesso em: 09 ago. 2025. Citado na página 27.

Oracle. *O que é Java?* 2024. Disponível em: <https://www.java.com/pt-BR/download/help/whatis_java.html>. Citado na página 20.

Oracle. *O que é MySQL?* 2024. Disponível em: <https://www.oracle.com/br/mysql/what-is-mysql/>. Citado 2 vezes nas páginas 24 e 74.

PIT. *PIT Mutation Testing*. 2024. Disponível em: <https://pitest.org/>. Citado na página 21.

POLITOWSKI, C.; MARAN, V. *Comparac¸ao de Performance entre PostgreSQL e MongoDB*. 2014. Citado na página 24.

PostgreSQL Global Development Group. *About PostgreSQL*. 2024. Disponível em: <https://www.postgresql.org/about/>. Citado na página 19.

Postman. *What is Postman?* 2024. Disponível em: <https://www.postman.com/product/what-is-postman/>. Citado na página 21.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional, 9ª edição*. Porto Alegre: AMGH, 2021. 1305 p. Citado 3 vezes nas páginas 22, 25 e 28.

Redgate Software. *Redgate Data Modeler (formerly Vertabelo)*. 2026. Disponível em: <https://www.red-gate.com/products/redgate-data-modeler/>. Citado na página 17.

RIVERO, L. et al. Implementando o gitflow para gerencia de configuração em um projeto de desenvolvimento de software Ágil: Um relato de experiência. *XII Computer on the Beach*, 2021. Citado na página 27.

SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. 2020. Disponível em: <https://scrumguides.org/scrum-guide.html>. Citado na página 22.

SonarSource. *SonarQube*. 2024. Disponível em: <https://www.sonarsource.com/products/sonarqube/>. Citado na página 21.

SOUZA, M. C. *Comunicação em tempo real para jogo educativo Beer Game*. 2023. Citado na página 23.

SqlDBM. *SqlDBM: Online Database Modeling*. 2026. Disponível em: <https://sqldbm.com/data-modeling/>. Citado na página 17.

StompJS. *StompJS*. 2023. Disponível em: <https://stomp-js.github.io/>. Citado na página 20.

Stryker Mutator. *Stryker Mutator: Test your tests with mutation testing*. 2024. Disponível em: <https://stryker-mutator.io/>. Citado na página 21.

VIEIRA, R. *UML — Unified Modeling Language*. 2015. Disponível em: <https://medium.com/operacionalti/uml-1f7b99dd15bb>. Citado na página 19.

VMware. *Spring Boot*. 2024. Disponível em: <https://spring.io/projects/spring-boot>. Citado 2 vezes nas páginas 20 e 21.

VMware. *Spring Data JPA*. 2024. Disponível em: <https://spring.io/projects/spring-data-jpa>. Citado na página 20.

VMware. *Spring Data MongoDB*. 2024. Disponível em: <https://spring.io/projects/spring-data-mongodb>. Citado na página 20.

VMware. *Spring Security*. 2024. Disponível em: <https://spring.io/projects/spring-security>. Citado 2 vezes nas páginas 20 e 27.

VMware. *WebSockets*. 2024. Disponível em: <https://docs.spring.io/spring-framework/reference/web/websocket.html>. Citado na página 20.

ZWEIHOFF, P.; NAUJOKAT, S.; STEFFEN, B. Pyro: Generating domain-specific collaborative online modeling environments. In: *Fundamental Approaches to Software Engineering (FASE 2019)*. [S.l.]: Springer, 2019. p. 101–115. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-030-16722-6_6>. Acesso em: 19 out. 2026. Citado na página 46.

Anexos

ANEXO A – Expansão do Caso de Uso

CSU-01: Logar

Caso de Uso: Logar

Identificador: CSU-01

Atores Envolvidos: Viewer, Editor, Owner

Sumário: Esta funcionalidade permite que o usuário previamente cadastrado autentique-se na plataforma, obtendo acesso às funcionalidades disponíveis.

Precondição:
- O usuário deverá estar previamente cadastrado no sistema.

Fluxo Principal:
- O usuário acessa a tela de login;
- O usuário insere suas credenciais (e-mail e senha);
- O usuário submete as informações;
- O usuário é redirecionado para a tela de listagem de projetos (CSU-03).

Exceção:
- Credenciais inválidas: Caso as credenciais informadas sejam inválidas, será mostrada uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário poderá acessar as funcionalidades que o perfil de sua conta permitir.

CSU-02: Criar Projeto

Caso de Uso: Criar Projeto

Identificador: CSU-02

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário crie um novo projeto na plataforma, tornando-se automaticamente o proprietário do projeto criado.

Precondição:
- O usuário deverá estar autenticado no sistema (CSU-01).

Fluxo Principal:
- O usuário acessa a opção de criar um novo projeto na interface;
- O usuário preenche as informações do projeto, como nome e descrição;
- O usuário confirma a criação do projeto;
- O projeto é criado e o usuário é redirecionado à tela de listagem de projetos (CSU-03).

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido ou campo obrigatório esteja em branco, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O novo projeto estará disponível na listagem de projetos (CSU-03) e o usuário poderá gerenciá-lo.

CSU-03: Listar Projetos

Caso de Uso: Listar Projetos

Identificador: CSU-03

Atores Envolvidos: Viewer, Editor, Owner

Sumário: Esta funcionalidade permite que o usuário autenticado visualize todos os projetos aos quais tem acesso como visualizador ou colaborador.

Precondição:
- O usuário deverá estar autenticado no sistema (CSU-01).

Fluxo Principal:
- Após a autenticação (CSU-01), o usuário é redirecionado à tela de listagem de projetos;
- Os projetos aos quais o usuário tem acesso são exibidos.

Exceção:
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário poderá selecionar um projeto para visualizar seu diagrama ER (CSU-06).
- Caso o usuário possua o perfil de Owner, poderá também criar projetos (CSU-02), editar projetos (CSU-04) e removê-los (CSU-05).

CSU-04: Editar Projeto

Caso de Uso: Editar Projeto

Identificador: CSU-04

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário edite as informações de um projeto existente ao qual é proprietário.

Precondição:
- O usuário deverá estar autenticado no sistema;
- O projeto a ser editado deverá existir na plataforma;
- O usuário deverá acessar o projeto pela tela de listagem (CSU-03).

Fluxo Principal:
- O usuário acessa a tela de listagem de projetos (CSU-03);
- O usuário seleciona o projeto que deseja editar;
- A tela de edição do projeto é exibida com os dados atuais;
- O usuário altera as informações desejadas do projeto;
- O usuário pode adicionar membros ao projeto (CSU-08) e gerenciar a lista de membros existentes (CSU-09);
- O usuário confirma as alterações;
- As alterações são persistidas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro ao usuário.
- Projeto não encontrado: Caso o projeto selecionado não exista na plataforma, será apresentada uma mensagem de erro indicando o ocorrido.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O projeto estará atualizado com as novas informações na plataforma.

CSU-05: Remover Projeto

Caso de Uso: Remover Projeto

Identificador: CSU-05

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário remova um projeto existente da plataforma, excluindo todos os seus dados associados.

Precondição:
- O usuário deverá estar autenticado no sistema;
- O projeto a ser removido deverá existir na plataforma;
- O usuário deverá acessar o projeto pela tela de listagem (CSU-03).

Fluxo Principal:
- O usuário acessa a tela de listagem de projetos (CSU-03);
- O usuário seleciona o projeto que deseja remover;
- O usuário aciona a opção de remoção do projeto;
- Uma janela de confirmação é exibida ao usuário;
- O usuário confirma a ação;
- O projeto e todos os seus dados associados são removidos da plataforma.

Exceção:
- Projeto não encontrado: Caso o projeto selecionado não exista na plataforma, será apresentada uma mensagem de erro indicando o ocorrido.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O projeto não estará mais disponível na plataforma e todos os seus dados terão sido excluídos.

CSU-06: Visualizar Diagrama

Caso de Uso: Visualizar Diagrama

Identificador: CSU-06

Atores Envolvidos: Viewer, Editor, Owner

Sumário: Esta funcionalidade permite que o usuário visualize o diagrama ER de um projeto ao qual tem acesso, podendo observar as entidades, atributos e relacionamentos modelados.

Precondição:
- O usuário deverá estar autenticado no sistema;
- O usuário deverá ter acesso ao projeto selecionado.

Fluxo Principal:
- O usuário acessa a tela de listagem de projetos (CSU-03);
- O usuário identifica o projeto cujo diagrama deseja visualizar;
- O usuário seleciona o projeto;
- O usuário é redirecionado à tela do diagrama ER do projeto;
- O diagrama é carregado e exibido ao usuário com todas as entidades, atributos e relacionamentos.

Exceção:
- Projeto não encontrado: Caso o usuário tente acessar um projeto que não existe na plataforma, será apresentada uma mensagem de erro indicando o ocorrido.
- Usuário sem acesso ao projeto: Caso o usuário tente acessar um projeto ao qual não tem permissão, será apresentada uma mensagem de erro e o usuário será redirecionado à tela de listagem de projetos (CSU-03).
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário poderá visualizar o diagrama ER com todas as entidades, atributos e relacionamentos cadastrados para o projeto.
- Caso o usuário possua o perfil de Editor ou Owner, poderá também editar o diagrama (CSU-07) e exportar o script DDL correspondente (CSU-23).

CSU-07: Editar Diagrama

Caso de Uso: Editar Diagrama

Identificador: CSU-07

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário realize alterações no diagrama ER de um projeto, podendo gerenciar tabelas, atributos e relacionamentos.

Precondição:
- O usuário deverá estar autenticado no sistema;
- O usuário deverá ser membro do projeto selecionado;
- O diagrama do projeto deverá estar sendo visualizado (CSU-06).

Fluxo Principal:
- Ao acessar o diagrama (CSU-06), o usuário dispõe das opções de edição habilitadas na interface;
- O usuário realiza as operações desejadas, podendo adicionar tabelas (CSU-13), editar tabelas (CSU-14), remover tabelas (CSU-15), adicionar relacionamentos (CSU-16), remover relacionamentos (CSU-17), mover elementos (CSU-18) ou importar um script SQL (CSU-24);
- As alterações são persistidas em tempo real e propagadas aos demais usuários conectados ao projeto.

Exceção:
- Usuário sem permissão de edição: Caso o usuário não possua o perfil de Editor no projeto, as opções de edição não estarão disponíveis e será exibida uma mensagem de erro caso a ação seja tentada.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O diagrama ER do projeto estará atualizado com as alterações realizadas pelo usuário.

CSU-08: Adicionar Membro

Caso de Uso: Adicionar Membro

Identificador: CSU-08

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário adicione um novo membro a um projeto, definindo seu perfil de acesso.

Precondição:
- O usuário deverá estar editando o projeto (CSU-04);
- O usuário a ser adicionado deverá estar previamente cadastrado na plataforma.

Fluxo Principal:
- Ao editar o projeto (CSU-04), o usuário seleciona a opção de adicionar um novo membro;
- O usuário define os dados e o perfil (role) do novo membro, podendo ser Viewer ou Editor;
- O membro é adicionado ao projeto com o perfil atribuído.

Exceção:
- Usuário não encontrado: Caso o usuário buscado não esteja cadastrado na plataforma, será exibida uma mensagem de erro ao usuário.
- Usuário já é membro: Caso o usuário já faça parte do projeto, será exibida uma mensagem informando que o usuário já é membro.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O novo membro estará listado no projeto com o perfil atribuído e poderá acessar as funcionalidades correspondentes ao seu perfil.

CSU-09: Listar Membros

Caso de Uso: Listar Membros

Identificador: CSU-09

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário visualize todos os membros de um projeto e seus respectivos perfis de acesso.

Precondição:
- O usuário deverá estar editando o projeto (CSU-04).

Fluxo Principal:
- Ao editar o projeto (CSU-04), os membros do projeto são exibidos ao usuário ;
- O usuário pode visualizar o perfil (role) de cada membro listado;
- Caso deseje, o usuário pode editar um membro (CSU-10) ou remover um membro (CSU-11).

Exceção:
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário poderá gerenciar os membros do projeto, podendo editá-los ou removê-los conforme necessário.

CSU-10: Editar Membro

Caso de Uso: Editar Membro

Identificador: CSU-10

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário edite as informações de um membro existente no projeto, como seu perfil de acesso.

Precondição:
- O usuário deverá estar na listagem de membros do projeto (CSU-09);
- O membro a ser editado deverá existir no projeto.

Fluxo Principal:
- Ao listar os membros (CSU-09), o usuário seleciona a opção de editar um membro;
- A interface de edição do membro é exibida com os dados atuais;
- O usuário altera os dados desejados;
- O usuário pode alterar o role do membro (CSU-12);
- As alterações são salvas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O membro estará atualizado com as novas informações no projeto.

CSU-11: Remover Membro

Caso de Uso: Remover Membro

Identificador: CSU-11

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário remova um membro de um projeto, revogando seu acesso à plataforma no contexto daquele projeto.

Precondição:
- O usuário deverá estar na listagem de membros do projeto (CSU-09);
- O membro a ser removido deverá existir no projeto.

Fluxo Principal:
- Ao listar os membros (CSU-09), o usuário seleciona a opção de remover um membro;
- Uma janela de confirmação é exibida ao usuário;
- O usuário confirma a ação;
- O membro é removido do projeto.

Exceção:
- Membro não encontrado: Caso o membro selecionado não exista no projeto, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O membro não terá mais acesso ao projeto na plataforma.

CSU-12: Alterar *Role*

Caso de Uso: Alterar *Role*

Identificador: CSU-12

Atores Envolvidos: Owner

Sumário: Esta funcionalidade permite que o usuário altere o perfil (*role*) de um membro existente no projeto, modificando suas permissões de acesso.

Precondição:
- O usuário deverá estar editando um membro do projeto (CSU-10);
- O membro a ter o *role* alterado deverá existir no projeto.

Fluxo Principal:
- Ao editar um membro (CSU-10), o usuário seleciona a opção de alterar o *role*;
- O usuário seleciona o novo perfil para o membro, podendo ser Viewer ou Editor;
- A alteração é confirmada;
- O membro passa a ter o novo perfil atribuído no projeto.

Exceção:
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O membro terá o novo perfil atribuído no projeto, com as permissões correspondentes ao *role* selecionado.

CSU-13: Adicionar Tabela

Caso de Uso: Adicionar Tabela

Identificador: CSU-13

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário adicione uma nova tabela (entidade) ao diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07).

Fluxo Principal:
- O usuário seleciona a opção de adicionar uma nova tabela na interface de edição;
- Uma nova tabela é criada no canvas do diagrama com um nome padrão;
- O usuário define o nome da tabela;
- O usuário pode adicionar atributos à tabela (CSU-19) e visualizar a listagem dos atributos cadastrados (CSU-20);
- As alterações são persistidas.

Exceção:
- Nome de tabela inválido: Caso o nome informado seja inválido ou esteja em branco, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- A nova tabela estará visível no diagrama ER do projeto.

CSU-14: Editar Tabela

Caso de Uso: Editar Tabela

Identificador: CSU-14

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário edite os dados de uma tabela existente no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- A tabela a ser editada deverá existir no diagrama.

Fluxo Principal:
- O usuário seleciona uma tabela existente no diagrama;
- A interface de edição da tabela é exibida com os dados atuais;
- O usuário altera o nome ou outras propriedades da tabela;
- O usuário pode adicionar novos atributos (CSU-19) e gerenciar os atributos existentes por meio da listagem (CSU-20);
- As alterações são persistidas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- A tabela estará atualizada no diagrama ER do projeto.

CSU-15: Remover Tabela

Caso de Uso: Remover Tabela

Identificador: CSU-15

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário remova uma tabela existente do diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- A tabela a ser removida deverá existir no diagrama.

Fluxo Principal:
- O usuário seleciona a tabela que deseja remover;
- O usuário aciona a opção de remoção;
- Uma janela de confirmação é exibida ao usuário;
- O usuário confirma a ação;
- A tabela e todos os relacionamentos a ela associados são removidos do diagrama;
- As alterações são persistidas.

Exceção:
- Tabela não encontrada: Caso a tabela selecionada não exista no diagrama, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- A tabela e seus relacionamentos são removidos do diagrama ER do projeto.

CSU-16: Adicionar Relacionamento

Caso de Uso: Adicionar Relacionamento

Identificador: CSU-16

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário adicione um relacionamento entre duas tabelas no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- Deverão existir pelo menos duas tabelas com chaves primárias no diagrama.

Fluxo Principal:
- O usuário seleciona a opção de adicionar um relacionamento na interface de edição;
- O usuário seleciona a tabela de origem do relacionamento;
- O usuário seleciona a tabela de destino do relacionamento;
- O usuário define a cardinalidade do relacionamento;
- O relacionamento é criado e exibido no diagrama conectando as duas tabelas;
- As alterações são persistidas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O relacionamento estará visível no diagrama ER conectando as duas tabelas selecionadas.

CSU-17: Remover Relacionamento

Caso de Uso: Remover Relacionamento

Identificador: CSU-17

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário remova um relacionamento existente entre tabelas no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- O relacionamento a ser removido deverá existir no diagrama.

Fluxo Principal:
- O usuário seleciona o relacionamento que deseja remover;
- O usuário aciona a opção de remoção;
- O relacionamento é removido do diagrama;
- As alterações são persistidas.

Exceção:
- Relacionamento não encontrado: Caso o relacionamento selecionado não exista no diagrama, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O relacionamento é removido do diagrama ER do projeto.

CSU-18: Mover Elemento no Diagrama

Caso de Uso: Mover Elemento no Diagrama

Identificador: CSU-18

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário reposicione tabelas e demais elementos no canvas do diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- O elemento a ser movido deverá existir no diagrama.

Fluxo Principal:
- O usuário seleciona o elemento que deseja reposicionar no canvas;
- O usuário arrasta o elemento até a posição desejada;
- O elemento é exibido na nova posição;
- A nova posição é persistida.

Exceção:
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O elemento é exibido na nova posição no diagrama ER do projeto.

CSU-19: Adicionar Atributo

Caso de Uso: Adicionar Atributo

Identificador: CSU-19

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário adicione um novo atributo a uma tabela no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- A tabela deverá existir no diagrama.

Fluxo Principal:
- Ao adicionar (CSU-13) ou editar (CSU-14) uma tabela, o usuário seleciona a opção de adicionar um atributo;
- O usuário preenche as informações do atributo;
- O usuário submete os dados do atributo;
- O atributo é adicionado à tabela no diagrama;
- As alterações são persistidas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro indicando qual campo está incorreto.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O novo atributo estará listado na tabela no diagrama ER do projeto.

CSU-20: Listar Atributos

Caso de Uso: Listar Atributos

Identificador: CSU-20

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário visualize todos os atributos de uma tabela no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- A tabela consultada deverá existir no diagrama.

Fluxo Principal:
- Ao adicionar (CSU-13) ou editar (CSU-14) uma tabela, os atributos cadastrados para aquela tabela são exibidos ao usuário ;
- O usuário pode visualizar todos os atributos com suas respectivas configurações;
- Caso deseje, o usuário pode editar um atributo (CSU-21) ou remover um atributo (CSU-22).

Exceção:
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário poderá gerenciar os atributos da tabela, podendo editá-los ou removê-los conforme necessário.

CSU-21: Editar Atributo

Caso de Uso: Editar Atributo

Identificador: CSU-21

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário edite as propriedades de um atributo existente em uma tabela do diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- O atributo a ser editado deverá existir na tabela.

Fluxo Principal:
- Ao listar os atributos de uma tabela (CSU-20), o usuário seleciona a opção de editar um atributo;
- A interface de edição do atributo é exibida com os valores atuais;
- O usuário altera os campos desejados;
- O usuário salva as alterações;
- O atributo é atualizado na tabela no diagrama;
- As alterações são persistidas.

Exceção:
- Dados inválidos: Caso algum dado informado seja inválido, será exibida uma mensagem de erro indicando qual campo está incorreto.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O atributo estará atualizado na tabela no diagrama ER do projeto.

CSU-22: Remover Atributo

Caso de Uso: Remover Atributo

Identificador: CSU-22

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário remova um atributo de uma tabela no diagrama ER do projeto.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07);
- O atributo a ser removido deverá existir na tabela.

Fluxo Principal:
- Ao listar os atributos de uma tabela (CSU-20), o usuário seleciona a opção de remover um atributo;
- Uma janela de confirmação é exibida ao usuário;
- O usuário confirma a ação;
- O atributo é removido da tabela no diagrama;
- As alterações são persistidas.

Exceção:
- Atributo não encontrado: Caso o atributo selecionado não exista na tabela, será exibida uma mensagem de erro ao usuário.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O atributo é removido da tabela no diagrama ER do projeto.

CSU-23: Exportar DDL

Caso de Uso: Exportar DDL

Identificador: CSU-23

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário exporte o diagrama ER do projeto na forma de um script Data Definition Language (DDL) em SQL.

Precondição:
- O usuário deverá estar visualizando um diagrama (CSU-06);
- O diagrama deverá possuir ao menos uma tabela cadastrada.

Fluxo Principal:
- Ao visualizar o diagrama (CSU-06), o usuário seleciona a opção de exportar o script DDL;
- O sistema gera o script DDL correspondente ao estado atual do diagrama ER;
- O arquivo gerado é disponibilizado para download;
- O usuário realiza o download do arquivo.

Exceção:
- Diagrama vazio: Caso o diagrama não possua nenhuma tabela cadastrada, será exibida uma mensagem informando que não há elementos para exportar.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O usuário possui um arquivo de script DDL SQL correspondente ao diagrama ER do projeto.

CSU-24: Importar SQL

Caso de Uso: Importar SQL

Identificador: CSU-24

Atores Envolvidos: Editor, Owner

Sumário: Esta funcionalidade permite que o usuário importe um script SQL/DDL para popular o diagrama ER do projeto de forma automática.

Precondição:
- O usuário deverá estar em modo de edição do diagrama (CSU-07).

Fluxo Principal:
- O usuário seleciona a opção de importar um script SQL na interface de edição;
- O usuário fornece o script SQL/DDL, seja por meio de upload de arquivo;
- O sistema faz o parse do script e gera as tabelas e relacionamentos correspondentes no diagrama;
- O diagrama é atualizado com a estrutura importada;
- As alterações são persistidas.

Exceção:
- Script SQL inválido: Caso o script fornecido contenha erros de sintaxe ou estrutura não suportada, será exibida uma mensagem de erro indicando o problema encontrado.
- Erro de instabilidade dos servidores: Caso algum erro ocorra nos servidores, será apresentada uma mensagem explicando que houve alguma falha de comunicação no sistema e encorajando o usuário a tentar novamente mais tarde.

Pós-Condição:
- O diagrama ER do projeto estará populado com as tabelas e relacionamentos definidos no script SQL importado.